

```
SQL> -- day-1.docx file exercise
SQL> -- 1. Display the structure of an EMP table
SQL> desc emp;
```

Name	Null?	Type
EMPNO	NOT NULL	NUMBER(4)
ENAME		VARCHAR2(10)
JOB		VARCHAR2(9)
MGR		NUMBER(4)
HIREDATE		DATE
SAL		NUMBER(7,2)
COMM		NUMBER(7,2)
DEPTNO		NUMBER(2)

```
SQL> -- 2. Display the structure of DEPT table
SQL> desc dept;
```

Name	Null?	Type
DEPTNO	NOT NULL	NUMBER(2)
DNAME		VARCHAR2(14)
LOC		VARCHAR2(13)

```
SQL> -- 3. Display all the records of EMP table
SQL> select * from emp;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

7934 MILLER

CLERK

7782 23-JAN-82

1300

10

14 rows selected.

SQL> -- 4. Display all the records of DEPT table

SQL> **select * from dept;**

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

SQL> -- 5. Display only Name of all employees

SQL> **select ename from emp;**

ENAME
SMITH
ALLEN
WARD
JONES
MARTIN
BLAKE
CLARK
SCOTT
KING
TURNER
ADAMS
JAMES
FORD
MILLER

14 rows selected.

SQL> -- 6. Display Employee's Name and salary

SQL> **select ename, sal from emp;**

ENAME	SAL
SMITH	800
ALLEN	1600

WARD	1250
JONES	2975
MARTIN	1250
BLAKE	2850
CLARK	2450
SCOTT	3000
KING	5000
TURNER	1500
ADAMS	1100
JAMES	950
FORD	3000
MILLER	1300

14 rows selected.

SQL> -- 7. Display only unique departments (deptno) from EMP table

SQL> select distinct(deptno) from emp;

DEPTNO
30
20
10

SQL> -- 8. Display employees whose name starts with 'J';

SQL> select * from emp where ename like 'J%';

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30

SQL> -- 9. Display all the employees' Date of Joining (HireDate) and Salary (Sal)

SQL> select hiredate, sal from emp;

HIREDATE	SAL
17-DEC-80	800
20-FEB-81	1600
22-FEB-81	1250
02-APR-81	2975

28-SEP-81	1250
01-MAY-81	2850
09-JUN-81	2450
19-APR-87	3000
17-NOV-81	5000
08-SEP-81	1500
23-MAY-87	1100
03-DEC-81	950
03-DEC-81	3000
23-JAN-82	1300

14 rows selected.

SQL> -- 10. Display all clerks (job), from EMP table

SQL> **select job from emp;**

```

JOB
-----
CLERK
SALESMAN
SALESMAN
MANAGER
SALESMAN
MANAGER
MANAGER
ANALYST
PRESIDENT
SALESMAN
CLERK
CLERK
ANALYST
CLERK

```

14 rows selected.

SQL> -- 11. Display all employees' name and salary whose salary is more than 2000

SQL> **select ename, sal from emp where sal > 2000;**

ENAME	SAL
JONES	2975
BLAKE	2850

CLARK	2450
SCOTT	3000
KING	5000
FORD	3000

6 rows selected.

SQL> -- 12. Display all employees who are not in department number (deptno) 30

SQL> **select * from emp where deptno != 30;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

8 rows selected.

SQL> -- 13. Display employees with their empno, ename and mgr (i.e manager's no)

SQL> **select empno, ename, mgr from emp;**

EMPNO	ENAME	MGR
7369	SMITH	7902
7499	ALLEN	7698
7521	WARD	7698
7566	JONES	7839
7654	MARTIN	7698
7698	BLAKE	7839
7782	CLARK	7839
7788	SCOTT	7566
7839	KING	
7844	TURNER	7698
7876	ADAMS	7788
7900	JAMES	7698
7902	FORD	7566
7934	MILLER	7782

14 rows selected.

SQL> -- 14. Display manager's number, job profile, department number and salary of employees Allen, Adams, Jones and Blake.

SQL> **select mgr, job, deptno, sal from emp where ename = 'ALLEN' OR ename = 'ADAMS' OR ename = 'JONES' OR ename = 'BLAKE';**

MGR	JOB	DEPTNO	SAL
7698	SALESMAN	30	1600
7839	MANAGER	20	2975
7839	MANAGER	30	2850
7788	CLERK	20	1100

4 rows selected.

SQL> -- 15. Display only unique Salary (sal) from EMP table

SQL> **select distinct(sal), ename from emp;**

SAL	ENAME
1600	ALLEN
1500	TURNER
5000	KING
1250	MARTIN
1250	WARD
1300	MILLER
800	SMITH
2450	CLARK
3000	SCOTT
1100	ADAMS
2850	BLAKE
2975	JONES
950	JAMES
3000	FORD

14 rows selected.

SQL> -- 16. Display the location of department number 30

SQL> **select deptno, loc from dept where deptno = 30;**

DEPTNO	LOC
--------	-----

30 CHICAGO

SQL> -- 17. Display the details of the department located in 'New York' city

SQL> **select * from dept where loc = 'NEW YORK';**

DEPTNO DNAME LOC

10 ACCOUNTING NEW YORK

SQL> -- 18. Display all the employees according to their names in sorted order

SQL> **select * from emp ORDER BY ENAME;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30

14 rows selected.

SQL> -- 19. Display all the employees who are not salesman nor the manager

SQL> **select * from emp where job NOT IN('SALESMAN', 'MANAGER');**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

7934	MILLER	CLERK	7782	23-JAN-82	1300		10
------	--------	-------	------	-----------	------	--	----

7 rows selected.

SQL> -- 20. Display the details of a clerk who is getting salary more than 1000 'MANAGER'

SQL> **select * from emp where job = 'CLERK' AND sal > 1000;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

SQL> -- 21. Display all the employees whose name starts with any alphabet between B and K

SQL> **select * from emp where ename between 'B%' AND 'K%';**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

SQL> -- 22. Display all employees whose name does not contain the character 'S'

SQL> **select * from emp where ename NOT like('%S%');**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

9 rows selected.

SQL> -- 23. Display all the employees whose salary is between 1000 and 2000 but not exact 1500

SQL> **select * from emp where sal between 1000 AND 2000 AND sal != 1500;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

SQL> -- 24. Select all the employees who are hired in the year 1981 (year 81)

SQL> **select * from emp where hiredate like '%81';**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

10 rows selected.

SQL> -- 25. Display all the employees who are hired in February (FEB) month

SQL> **select * from emp where hiredate like '%FEB%';**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30

SQL> -- 26. Display all the employees according to their seniority of joining

SQL> **select * from emp order by hiredate;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20

7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7839	KING	PRESIDENT		17-NOV-81	5000		10
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20

14 rows selected.

SQL> -- 27. Display the employees who are either 'Analyst', 'President' or 'Manager'
SQL> **select * from emp where job in('ANALYST', 'PRESIDENT', 'MANAGER');**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

6 rows selected.

SQL> -- 28. Display all the Salesman joined in month of September and getting salary more than or equal to 1500
SQL> **SELECT * from emp where hiredate like '%FEB%' AND sal >= 1500;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30

SQL> -- 29. Display the job type of an employee of department number 30 and getting salary more than 1500
SQL> **SELECT * from emp where deptno=30 AND sal >= 1500**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30

SQL> -- 30. Display the unique job titles of employees in department number 20

SQL> **select distinct(job) from emp where deptno=20;**

JOB

CLERK

MANAGER

ANALYST

SQL> -- 31. Display all the employees in a sorted order to their salary highest to lowest, hiredate latest to oldest.

SQL> **select * from emp order by sal desc;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7369	SMITH	CLERK	7902	17-DEC-80	800		20

14 rows selected.

SQL> **select * from emp order by hiredate desc;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7839	KING	PRESIDENT		17-NOV-81	5000		10
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30

7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7369	SMITH	CLERK	7902	17-DEC-80	800		20

14 rows selected.

SQL> -- 32. Display all the employees in a sorted order of their job title and salary.

SQL> **select * from emp order by job desc;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7839	KING	PRESIDENT		17-NOV-81	5000		10
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20

14 rows selected.

SQL> **select * from emp order by sal desc;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30

7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7369	SMITH	CLERK	7902	17-DEC-80	800		20

14 rows selected.

SQL> -- 33. Display employees in a sorted order of their salary who are clerk.

SQL> **select * from emp where job = 'CLERK' order by sal;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

SQL> -- 34. Display all the managers according to their salary highest to lowest

SQL> **select * from emp where job = 'MANAGER' order by sal desc;**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10

3 rows selected.

SQL> -- 35. Display all the employees who have joined either on 3rd or 23rd date of any month in any year.

SQL> **select * from emp where hiredate like '03%' OR hiredate like '23%';**

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		

```
SQL> -- day-2.docx file exercise
```

```
SQL> -- Topics:
```

1. Oracle Data Types: Char, Varchar, Date, Number, Long, Raw/ Long Raw
2. Data Constraints: Table Level and Row Level Constraint:
 - a. I/O Constraints: Unique, Primary Key, Reference Key
 - b. Business Rule Constraints: NOT NULL, Check, default
3. Creating Table
4. Inserting Data into Table
Viewing Data in the Table.

Exercise

Create below given table called "Client_Master" with the constrains and insert minimum 5 records in it.

Column	Datatype and Size	Constraints
C_No	Varchar (4)	Primary Key
Client_Name	Varchar (25)	Not Null
Pincode	Number (6)	Not Null
Birth_Date	Date	Not Null
Occupation	Varchar (15)	Not Null

Add the following constraints with the table fields:

1. Client_No must start with the capital letter 'C' only.

2. Pincode must be of exactly 6-digit number.

3. Sample records are as below

C_No	Client_Name	Pincode	Birth_Date	Occupation
C001	Kajal Oza	380036	14-july-1987	Manager
C002	Vraj Shah	380058	20-nov-1986	CEO
C003	Dhyan Dave	387903	17-June-1988	Doctor

C004	Mira Vaid	398006	5-jan-1976	Professor
C005	Preeti Patel	390040	28-feb-1971	Clerk

```

Create table Client_Master(
    C_No varcahr2(4) CHECK(C_No LIKE 'C%') PRIMARY KEY,
    Client_Name varchar2(25) NOT NULL,
    Pincode Number(6) CHECK(LENGTH(Pincode)=6) NOT NULL,
    Birth_Date Date NOT NULL,
    Occupation varchar2(15) NOT NULL
);

INSERT INTO Client_Master VALUES ('C001', 'Kajal Oza', 380036, '14-july-1987', 'Manager');
INSERT INTO Client_Master VALUES ('C002', 'Vraj Shah', 380058, '20-nov-1986', 'CEO');
INSERT INTO Client_Master VALUES ('C003', 'Dhyan Dave', 387903, '17-june-1988', 'Doctor');
INSERT INTO Client_Master VALUES ('C004', 'Mira Vaid', 398006, '5-jan-1976', 'Professor');
INSERT INTO Client_Master VALUES ('C005', 'Preeti Patel', 390040, '28-feb-1971', 'Clerk');

```

Create following tables with constraints and records as shown in the example below.

Table: **Product_Master**

P_No	Desc	Profit_Per	Unit	Qty_Hand	ReOrder	Sell_P	Cost_P
P001	T-Shirt	5	Piece	200	50	350	250
P002	Jeans	6	Piece	150	40	500	350
P003	Skirt	6	Piece	100	50	350	200
P004	Saree	3	Piece	100	20	800	600

Create Table and Insert minimum 10 records in **Salesman_Master** Table as shown below. (Sample data are shown for your reference):

S_No	Name	Add1	Add2	City	Pin Code	State	Salary	Target	Sales
S001	Aman	3, Dev Appt	Vaghodia	Baroda		Gujarat	5000	100	50
S002	Omkar			Bhopal		MP	4500	200	150
S003	Raj	B-104	Verli	Mumbai	400002	Maharashtra	5500	200	200

S004	Ashish	1, Smruti	Ghoddol	Surat		Gujarat	4500	150	100
------	--------	-----------	---------	-------	--	---------	------	-----	-----

```
SQL> create table salesman_master(
2   s_no varchar2(5) check (s_no like 'S%') primary key,
3   name varchar2(20) not null,
4   add1 varchar2(10),
5   add2 varchar2(15),
6   city varchar2(10) not null,
7   pincode number(6),
8   state varchar2(15) not null,
9   salary number(10) not null,
10  target number(5) not null,
11  sales number(5) not null);
```

Create Sales_Order and Sales_Order_Details table as shown below. Insert minimum 10 records in it.

Sales_Order

Fields	Description
Order_No	Primary Key, Varchar (4), Starts with 'O'
Client_No	References Client_Master
Order_Date	Date
Salesman_No	Refers Salesman_Master
Delivery_Type	Free or Paid. Use Char(1) i.e 'F' or 'P'
Bill_Paid	Yes ('Y') or No ('N')
Delivery_Date	Date
Order_Status	'In Process', 'Pending', 'Fulfilled'

```
SQL> CREATE TABLE Sales_Order (
2   Order_No      VARCHAR2(4) PRIMARY KEY CHECK (Order_No LIKE 'O%'),
3   Client_No     VARCHAR2(5),
4   Order_Date    DATE,
5   Salesman_No   VARCHAR2(4) REFERENCES Salesman_Master(S_No),
6   Delivery_Type CHAR(1) CHECK (Delivery_Type IN ('F','P')),
7   Bill_Paid     CHAR(1) CHECK (Bill_Paid IN ('Y','N')),
8   Delivery_Date DATE,
9   Order_Status  VARCHAR2(20) CHECK (Order_Status IN ('In Process','Pending','Fulfilled'))
10 );
```

Sales_Order_Details: Order_No and Product_No are the composite PK

Fields	Description
Order_No	Refers Sales_Order Table
Product_No	Refers Product_Master
Qty_Ordered	Positive integer
Qty_Dispatched	Positive integer
Product_Rate	Product Selling Price * Qantity Delivered

```
SQL> CREATE TABLE Sales_Order_Details (  
2     Order_No      VARCHAR2(4),  
3     Product_No    VARCHAR2(4),  
4     Qty_Ordered   NUMBER(6) CHECK (Qty_Ordered > 0),  
5     Qty_Dispatched NUMBER(6) CHECK (Qty_Dispatched >= 0),  
6     Product_Rate  NUMBER(10,2) CHECK (Product_Rate >= 0),  
7     PRIMARY KEY (Order_No, Product_No),  
8     FOREIGN KEY (Order_No) REFERENCES Sales_Order(Order_No),  
9     FOREIGN KEY (Product_No) REFERENCES Product_Master(P_No)  
10 );
```

Create following tables with constraints given below. Also enter minimum 5 records in Entrance_Test table and 10 records in each of other two tables:

```
APPLICANT (AID, A_Name, Address, B_Date)  
ENTRANCE_TEST (ET_ID, ET_Name, Max_Score)  
E_TEST_DETAILS (AID, ETID, ETest_Date, Score)
```

- AID: Applicant ID must start with a letter 'A' and should be of length 4, It will be the foreign key for E_TEST_DETAILS Table
- Max Score of any exam must not be greater than 100
- ET_ID is the entrance test ID, starts with 'T' and is length of 4, referred as Foreign Key for E_TEST_DETAILS Table

```
SQL> CREATE TABLE Applicant (  
2     AID      VARCHAR2(4) PRIMARY KEY CHECK (AID LIKE 'A%'),  
3     A_Name   VARCHAR2(30) NOT NULL,  
4     Address  VARCHAR2(50),  
5     B_Date   DATE
```

```
6 );
```

```
SQL> CREATE TABLE Entrance_Test (  
2     ET_ID      VARCHAR2(4) PRIMARY KEY CHECK (ET_ID LIKE 'T%'),  
3     ET_Name    VARCHAR2(30) NOT NULL,  
4     Max_Score  NUMBER(3) CHECK (Max_Score <= 100)  
5 );
```

```
SQL> CREATE TABLE ETest_Details (  
2     AID        VARCHAR2(4),  
3     ETID       VARCHAR2(4),  
4     ETest_Date DATE,  
5     Score      NUMBER(3) CHECK (Score >= 0),  
6     PRIMARY KEY (AID, ETID),  
7     FOREIGN KEY (AID) REFERENCES Applicant(AID),  
8     FOREIGN KEY (ETID) REFERENCES Entrance_Test(ET_ID)  
9 );
```

Create following tables with constraints given below. Also enter minimum 5 records in Distributor table and Item table. Enter 10 records in Dist_Item table.

Distributor (Dno, DName, City, Phone)

Item (Item_No, Item_Name, Price, Weight)

Dist_Item (Dno, Item_No, Qty, Date)

- Dno is the PK and referred as FK for 3rd table
- Item_No starts with 'I' and maximum length of 5

Price, weight, Qty (quantity) must be greater than 0 and not null.

```
SQL> CREATE TABLE Distributor (  
2     Dno      VARCHAR2(4) PRIMARY KEY,  
3     DName    VARCHAR2(30) NOT NULL,  
4     City     VARCHAR2(30),  
5     Phone    VARCHAR2(12)  
6 );
```

```
SQL> CREATE TABLE Item (  
2     Item_No   VARCHAR2(5) PRIMARY KEY,  
3     Item_Name VARCHAR2(30) NOT NULL,  
4     Price     NUMBER(8,2) CHECK (Price > 0),  
5     Weight    NUMBER(8,2) CHECK (Weight > 0),  
6 );
```

```
2      Item_No   VARCHAR2(5) PRIMARY KEY CHECK (Item_No LIKE 'I%'),
3      Item_Name VARCHAR2(30) NOT NULL,
4      Price     NUMBER(8,2) NOT NULL CHECK (Price > 0),
5      Weight    NUMBER(8,2) NOT NULL CHECK (Weight > 0)
6  );
```

```
SQL> CREATE TABLE Dist_Item (
2      Dno       VARCHAR2(4),
3      Item_No   VARCHAR2(5),
4      Qty       NUMBER(6) NOT NULL CHECK (Qty > 0),
5      Order_Date DATE,
6      PRIMARY KEY (Dno, Item_No),
7      FOREIGN KEY (Dno) REFERENCES Distributor(Dno),
8      FOREIGN KEY (Item_No) REFERENCES Item(Item_No)
9  );
```

```
SQL> -- day-3.docx file exercise
```

```
SQL> -- Topics:
```

1. Creating a Table from a Table
2. Inserting Data into a Table from Another Table
3. Deleting records from the table: Using Delete and Truncate operations
4. Destroying Table

```
SQL> -- 1. Remove all Sales_Orders whose QtyOrdered = 1 in "Sales_Order_Detail" table
```

```
SQL> delete from sales_order_details where QTY_ORDERED = 1;
```

```
1 row deleted.
```

```
SQL> -- 2. Modify the OrderStatus to "Fulfilled" where the OrderStatus is "In Process" in Sales_Order Table
```

```
SQL> UPDATE Sales_Order SET ORDER_STATUS='Fulfilled' WHERE ORDER_STATUS='In Process';
```

3 rows updated.

SQL> -- 3. Increase the sell price of saree by Rs. 20

SQL> **update PRODUCT_MASTER set SELL_P=SELL_P+20 WHERE DESCRIPTION='SAREE';**

1 row updated.

SQL> -- 4. Delete all Salesman from Salesman_Master table whose salary is more than 3500

SQL> **DELETE FROM Salesman_Master WHERE SALARY>3500;**

ERROR at line 1: ORA-02292: integrity constraint (MCAB250013.SYS_C0032324) violated - child record found

SQL> -- here, Salesman_Master table relation with SALES_ORDER. SALES_ORDER table store salesman_master table S_NO in column SALESMAN_NO than without remove column (SALES_ORDER.SALESMAN_NO) constraint query 4 not work

SQL> -- now first remove SALES_ORDER column SALESMAN_NO constraint

SQL> **alter table SALES_ORDER drop constraints SYS_C0032324;**

Table altered.

SQL> **DELETE FROM Salesman_Master WHERE SALARY>3500;**

10 rows deleted.

SQL> -- 5. Delete all Products from Product_master where Quantity on Hand (Qty_Hand) is less than 50

SQL> **delete from PRODUCT_MASTER where qty_hand > 100;**

ERROR at line 1: ORA-02292: integrity constraint (MCAB250013.SYS_C0032330) violated - child record found

SQL> **alter table Sales_Order_Details drop constraints SYS_C0032330;**

Table altered.

SQL> **delete from PRODUCT_MASTER where qty_hand > 100;**

7 rows deleted.

```
SQL> -- 6. Change the details of the Client 'C003' as pincode to 387903 and Occupation is 'Manager'
```

```
SQL> update client_master set pincode=387903, OCCUPATION='Manager' where c_no='C003';
```

1 row updated.

```
SQL> -- 7. Remove the product details column from Product_Master Table
```

```
SQL> alter table PRODUCT_MASTER drop column DESCRIPTION;
```

Table altered.

```
SQL> -- 8. Add column City and State in Client_Master table
```

```
SQL> alter table Client_Master add (City varchar2(20), State varchar2(20));
```

Table altered.

```
SQL> -- 9. Increase the size of the Price Column
```

```
SQL> alter table ITEM modify price number(12, 3);
```

Table altered.

```
SQL> -- 10. Add a column Mobile_No of type Number, in the Client_Master Table
```

```
SQL> alter table Client_Master add Mobile_No number(10);
```

Table altered.

```
SQL> -- 11. Add constraint to the Mobile_No column;
```

```
SQL> alter table Client_Master add constraints MOBILE_NO CHECK(LENGTH(MOBILE_NO) = 10);
```

Table altered.

```
SQL> -- 12. Change the name of Salesman_Master table to "SMan_Mst"
```

```
SQL> rename Salesman_Master TO SMan_Mst;
```

Table renamed.

```
SQL> -- 13. Add a column of Experience in the Faculty_Master Table
```

```
SQL> alter table Salesman_Master add Experience varchar2(30);
```

Table altered.

SQL> -- 14. Change the name of the column Address to Area in Faculty_Master Table

SQL> alter table Salesman_Master rename column add1 to area;

Table altered.

SQL> -- 15. Modify the Applicant Table name to Applicant_Master

SQL> rename Applicant to Applicant_Master;

Table renamed.

SQL> -- 16. Add a column "Hobby" in Applicant_Master table.

SQL> alter table Applicant_Master add Hobby varchar2(30);

Table altered.

SQL> -- 17. Add a column "Location" in the "ETest_Detail" table. Keep it varchar(10)

SQL> alter table ETEST_DETAILS add Location varchar(10);

Table altered.

SQL> -- 18. Modify the column "Location" and increase the size upto 25 characters.

SQL> alter table ETEST_DETAILS modify Location varchar(25);

Table altered.

SQL> -- 19. Change the field "Address" to "City" in Applicant_Master Table

SQL> alter table Applicant_Master rename column ADDRESS to City;

Table altered.

SQL> -- 20. Change the name of the Item table to "Item_Master"

SQL> rename Item to Item_Master;

Table renamed.

```
SQL> -- day-4.docx file exercise
```

```
SQL> -- Topics:
```

1. Updating the contents of a Table
2. Remove all the rows and selected rows from the Table.
3. Modifying the Structure of a Table: Add a Column, Drop a Column, Modify existing Column
4. Renaming a Table

```
SQL> -- 1. Create a table called "Clients" from the "Client_Master" table.
```

```
SQL> create table Clients as select * from Client_Master where 1=2;
```

Table created.

```
SQL> -- 2. Insert all records of "Client_Master" into "Clients"
```

```
SQL> insert into clients select * from CLIENT_MASTER
```

4 rows created.

```
SQL> -- 3. Create table "Products" from "Product_master"
```

```
SQL> create table Products as select * from Product_master where 5=3;
```

Table created.

```
SQL> -- 4. Copy only selected Product's information into "Products" table.
```

```
SQL> insert into Products select * from Product_master where QTY_HAND=100;
```

2 rows created.

```
SQL> -- 5. Create table "Salesman" from "Salesman_Master" where "Salesman" table contains Fields: Salesman number,  
Name, Address1, City, Target_Get, Achieved_Target
```

```
SQL> create table Salesman as select s_no,name,add1 as Address1,city,target as Achieved_Target from Salesman_Master;
```

Table created.

```
SQL> -- 6. Insert all records for the fields in new table from "Salesman_Master" table
```

```
SQL> insert into Salesman select s_no,name,add1 as Address1,city,target as Achieved_Target from Salesman_Master;
```

10 rows created.

```
SQL> -- 7. Create an "Order_Master" table form "Sales_Order" Table
```

```
SQL> create table Order_Master as select * from Sales_Order where 4=5;
```

Table created.

```
SQL> -- 8. Insert appropriate records in "Order_Master" from Sales_Order" table
```

```
SQL> insert into Order_Master select * from Sales_Order where Order_Status>'Fulfilled';
```

4 rows created.

```
SQL> -- 9. Create "Order_Detail" table from "Sales_Order_Detail" table.
```

```
SQL> create table Order_Detail as select * from Sales_Order_Details where 2=3;
```

Table created.

```
SQL> -- 10. Insert few selected records in "Order_Detail" from Sales_Order_Detail" table.
```

```
SQL> insert into Order_Detail select * from Sales_Order_Details where ORDER_NO in('O001', 'O002', 'O003', 'O004', 'O005');
```

5 rows created.

```
SQL> -- 11. Destroy tables: Client_Master, Product_Master and Salesman_Master table using Truncate
```

```
SQL> truncate table Client_Master;
```

Table truncated.

```
SQL> select * from Client_Master;
```

no rows selected

```
SQL> truncate table Product_Master;
```

Table truncated.

```
SQL> truncate table Salesman_Master;
```

Table truncated.

```
SQL> -- 12. Destroy tables: "Sales_Order" and Sales_Order_Details using drop operation.
```

```
SQL> drop table Sales_Order cascade constraints; -- if table have constraint than use 'cascade constraints' keyword to drop table
```

Table dropped.

```
SQL> drop table Sales_Order_Details;
```

Table dropped.

```
SQL> -- 14. Copy only the structure of Applicant Table into Applicant_Master
```

```
SQL> -- question 13 not exist in day-4.docx
```

```
SQL> create table Applicant_Master as select * from APPLICANT where 1=2;
```

Table created.

```
SQL> -- 15. Create a new table called Course_Master having same fields as Entrance_Test table but change the column name as Course_Code, Course_Name and Course_Credit
```

```
SQL> create table Course_Master as select et_id as Course_Code, et_name as Course_Name, MAX_SCORE as Course_Credit from ENTRANCE_TEST;
```

Table created.

```
SQL> -- 16. Insert new data in Course_Master table.
```

```
SQL> insert into Course_Master VALUES ('T006','Java',80);
```

1 row created.

```
SQL> -- 17. Create a view of Entrance_Test_Details table having only 2 columns
```

```
SQL> create or replace view ETEST_DETAILS_view as select aid,etid from ETEST_DETAILS;
```

View created.

```
SQL> -- 18. Create a view of Applicant Table
```

```
SQL> create or replace view APPLICANT_view as select * from APPLICANT;
```

View created.

```
SQL> -- 19. Change the record (Column values) in the view table.
```

```
SQL> update APPLICANT_view set A_NAME='Dip' where A_NAME='Deepa';
```

1 row updated.

```
SQL> -- 20 Remove the view.
```

```
SQL> drop view APPLICANT_view;
```

View dropped.

```
SQL> -- day-5.docx file exercise
```

```
SQL> -- Topics:
```

Logical operators like: AND, OR operator, Not Operator

Aggregate, Math and String functions: MIN, COUNT, MAX, AVG, SUM, ABS, POWER, ROUND, SQRT, EXP, MOD, TRUNC, FLOOR, CEIL, LOWER, INITCAP, UPPER, SUBSTR, ASCII, COMPOSE, TRANSLATE, LENGTH, LTRIM, RTRIM, TRIM, LPAD, RPAD, SYSDATE , EXTRACT (), DATE FORMAT functions TO_DATE (), TO_CHAR ()

```
SQL> -- 1. Display all the clients whose age is more than 25 years but less than 50 years
```

```
SQL> select CLIENT_NAME, (EXTRACT(YEAR from SYSDATE) - EXTRACT(YEAR from BIRTH_DATE)) as AGE from CLIENTS WHERE  
(EXTRACT(YEAR from SYSDATE) - EXTRACT(YEAR from BIRTH_DATE)) BETWEEN 25 AND 50;
```

CLIENT_NAME	AGE
Kajal Oza	38
Vraj Shah	39
Dhyan Dave	37
Kajal Oza	38
Vraj Shah	39
Dhyan Dave	37

6 rows selected.

SQL> -- 2. Display all the Salesman from Salesman_Master table where "Sale" is more than his "Target" and "Target" is more than 100

SQL> select * from Salesman_Master where sales>target AND target>100;

S_NO	NAME	ADD1	ADD2	CITY	PINCODE	STATE	SALARY	TARGET
S011	Devendra	Block-12	Park Street-2	Kolkata	700019	UP	6000	210

SQL> -- 3. Find the Salesman who are neither from Baroda or from Surat.

SQL> select * from Salesman_Master where city NOT IN('BARODA', 'SURAT');

S_NO	NAME	ADD1	ADD2	CITY	PINCODE	STATE	SALARY	TARGET
S002	OMKAR	NULL	NULL	BHOPAL		MP	4500	200
S003	RAJ	B-104	VERLI	MUMBAI	400002	MAHARASHTRA	5500	200
S005	Meera	10, Lotus	Alkapuri	Vadodara	390007	Gujarat	6000	300
S006	Karan	Sector-17	Navrangpura	Ahmedabad	380009	Gujarat	5500	220

S007	Priya	L-12	MG Road	Pune	411001 Maharashtra	5200	180
160							
S008	Dev	House-22	Civil Lines	Delhi	110054 Delhi	5800	250
200							
S009	Sahil	Flat-4	Indira Nagar	Lucknow	226016 UP	4900	160
140							
S010	Nidhi	Block-7	Park Street	Kolkata	700016 WB	5300	210
190							
S011	Devendra	Block-12	Park Street-2	Kolkata	700019 UP	6000	210
500							

9 rows selected.

SQL> -- 4. Display total number of clients using Client_Master Table.

SQL> select count(*) from client_master;

```

COUNT(*)
-----
          5

```

SQL> -- 5. Display the highest salary a salesman is getting.

SQL> select max(SALARY) from salesman_master;

```

MAX(SALARY)
-----
        6000

```

SQL> -- 6. Display all item names in upper case letters only

SQL> select upper(ITEM_NAME) from ITEM;

```

UPPER(ITEM_NAME)
-----
RICE
WHEAT
SUGAR
OIL
SALT

```

SQL> -- 7. Display current date and time

SQL> select to_char(SYSDATE, 'DD-MM-YYYY HH:MI:SS') from dual;

SYSDATE

14-SEP-25 12:30:35

SQL> -- 8. Display average target given to the salesman.

SQL> select avg(TARGET) from salesman_master;

AVG(TARGET)

194.166667

SQL> -- 9. Display the Birth Date (DOB in Student_Master Table) in a new format. (Eg. February 12, 1998)

SQL> select to_char(dob, 'Month dd, yyyy') as Birth Date from Student_Master;

Birth Date

February 12, 1998

May 23, 2000

November 15, 1997

August 30, 1999

SQL> -- 10. Display Date of Joining (DOJ, Faculty_Master) of all faculties in DD/MM/YY format

SQL> select to_char(doj, 'DD/MM/YY') as DOJ from Faculty_Master;

DOJ

15/06/10

01/09/12

20/01/15

05/11/18

SQL> -- 11. Display Only Birth Date and Month of all the students from Student_Master Table

SQL> select to_char(dob, 'DD/MM') as DD_MM from Student_Master;

```
DD_MM
-----
12/02
23/05
15/11
30/08
```

```
SQL> -- 12. Count the total number of employees.
```

```
SQL> select count(*) from EMPLOYEE_MASTER;
```

```
COUNT(*)
-----
4
```

```
SQL> -- 13. Calculate the average salary of all the employees.
```

```
SQL> select avg(SALARY) from EMPLOYEE_MASTER;
```

```
AVG(SALARY)
-----
16750
```

```
SQL> -- 14. Determine the maximum and minimum salary. Rename the output as max_salary and min_salary respectively.
```

```
SQL> select max(SALARY) as max_salary, min(SALARY) as min_salary from EMPLOYEE_MASTER;
```

```
MAX_SALARY MIN_SALARY
-----
22000      12000
```

```
SQL> -- 15. Count the number of employees having salary less than or equal to 15000.
```

```
SQL> select count(EMPLOYEE_NAME) from EMPLOYEE_MASTER where SALARY <= 15000;
```

```
COUNT(EMPLOYEE_NAME)
-----
2
```

```
SQL> -- 16. List the details of Employee month wise in DD/MM/YY format.
```

```
SQL> select EMPLOYEE_NAME, SALARY, to_char(JOINING_DATE, 'DD/MM/YY') from EMPLOYEE_MASTER order by extract(month from JOINING_DATE);
```

EMPLOYEE_NAME	SALARY	TO_CHAR (
Jill	22000	15/01/21
Adam	15000	10/04/19
Eve	12000	23/07/20
Sam	18000	05/12/18

```
SQL> -- 17. List the DOB in the format 'DD-Month-YY' eg. 12-February-91
```

```
SQL> select to_char(JOINING_DATE, 'DD-Month-YY') from EMPLOYEE_MASTER;
```

TO_CHAR (JOINING
10-April -19
23-July -20
05-December -18
15-January -21

```
SQL> -- day-5 Question 2 queries based on tables: distributors, item and dist_item
```

```
SQL> -- 1. Count total distributors
```

```
SQL> select count(*) from DISTRIBUTOR;
```

COUNT (*)
5

```
SQL> -- 2. Find the total price of all items
```

```
SQL> select sum(price) as total from item;
```

TOTAL
265

```
SQL> -- 3. Find the item wise total quantity of each item
SQL> select item_no, sum(QTY) from DIST_ITEM group by ITEM_NO;
```

ITEM_	SUM(QTY)
I001	220
I002	230
I003	120
I004	150
I005	450

```
SQL> -- 4. Display all items whose name starts with "S"
SQL> select * from item where ITEM_NAME like 'S%';
```

ITEM_	ITEM_NAME	PRICE	WEIGHT
I003	Sugar	35	3
I005	Salt	20	1

```
SQL> -- 5. Display Distribution details of all items with month (from the date) only.
SQL> select d.dno,d.dname,d.city,d.phone,dt.item_no,dt.qty,to_char(dt.order_date, 'Month') as order_month from
DISTRIBUTOR d JOIN DIST_ITEM dt ON d.dno=dt.dno;
```

DNO	DNAME	CITY	PHONE	ITEM_	QTY	ORDER_MONTH
D001	ABC Distributors	Baroda	9876543210	I001	100	September
D001	ABC Distributors	Baroda	9876543210	I002	80	September
D002	Shree Suppliers	Surat	9123456780	I003	50	September
D002	Shree Suppliers	Surat	9123456780	I004	60	September
D003	Om Traders	Mumbai	9988776655	I005	200	September
D003	Om Traders	Mumbai	9988776655	I001	120	September
D004	Global Supply	Delhi	9012345678	I002	150	September
D004	Global Supply	Delhi	9012345678	I003	70	September
D005	FastMart	Pune	9090909090	I004	90	September
D005	FastMart	Pune	9090909090	I005	250	September

10 rows selected.

SQL> -- 6. Display the distributors whose city name starts with "B"

SQL> select * from DISTRIBUTOR where city like 'B%';

DNO	DNAME	CITY	PHONE
D001	ABC Distributors	Baroda	9876543210

SQL> -- 7. Find the highest weight of an item

SQL> select max(WEIGHT) from item;

MAX(WEIGHT)
5

SQL> select ITEM_NAME, WEIGHT from item where WEIGHT = (select max(WEIGHT) from item);

ITEM_NAME	WEIGHT
Rice	5

SQL> -- day-6.docx file exercise

SQL> -- Topics: Select data using: Order By, Group by and Having clause

SQL> -- Question-1 tables: APPLICANT, ENTRANCE_TEST and ETEST_DETAILS

SQL> -- 1. How many applicants have appeared for each test?

SQL> select etid, count(aid) from ETEST_DETAILS group by (etid);

ETID	COUNT(AID)
T002	2
T005	2
T001	2
T003	2
T004	2

SQL> -- 2. Display highest score for each test

SQL> **select etid, max(score) from ETEST_DETAILS group by (etid);**

ETID	MAX(SCORE)
T002	70
T005	75
T001	90
T003	75
T004	95

SQL> -- 3. Display applicant's ID who appeared for more than 3 tests

SQL> **select aid, count(etid) from ETest_Details group by (aid) HAVING count(etid)>3;**

AID	COUNT(ETID)
A001	4

SQL> -- 4. Calculate applicant's average score across all test they have appeared in

SQL> **select etid, avg(score) from ETest_Details group by etid;**

ETID	AVG(SCORE)
T002	70
T005	72
T003	76.6666667
T001	85
T004	90

SQL> -- 5. Display number of applicants by city

SQL> select city, count(aid) as number_a from Applicant group by city;

CITY	NUMBER_A
Jaipur	1
Delhi	1
Mumbai	1
Baroda	2
Kolkata	1
Lucknow	1
Nagpur	1
Surat	1
Chennai	2
Pune	1

10 rows selected.

SQL> -- 6. Display ETID and Average score where average score is more than 50.

SQL> -- asper query no 4 all is more than 50 than we do for more than 80

SQL> select etid, avg(score) from ETEST_DETAILS group by etid having avg(score)>80;

ETID	AVG(SCORE)
T001	85
T004	90

SQL> -- 7. Count date wise total entrance test to be held.

SQL> SELECT TRUNC(etest_date) as test_date, COUNT(*) AS total_test FROM ETEST_DETAILS GROUP BY TRUNC(etest_date);

TEST_DATE	TOTAL_TEST
20-AUG-25	1
23-AUG-25	2
21-AUG-25	2
27-AUG-25	1
25-AUG-25	2

26-AUG-25	1
15-AUG-25	1
18-AUG-25	1
22-AUG-25	1
17-AUG-25	1

10 rows selected.

SQL> -- Question-2 tables: Distributor, Item and Dist_Item

SQL> -- 1. Display city wise total number of distributors

SQL> select city, count(city) from Distributor group by city;

CITY	COUNT(CITY)
Delhi	1
Mumbai	1
Baroda	2
Surat	1
Pune	2

SQL> -- 2. List the distributors by who distributed more than 50 items in month of July

SQL> select dno, sum(qty) from Dist_Item where to_char(ORDER_DATE, 'MM')=8 group by dno having SUM(QTY) > 120;

DNO	SUM(QTY)
D001	180
D003	320
D004	150

SQL> -- 3. List Item wise total quantity delivered and display items having more than 5000 Qty delivered

SQL> select Item_No, sum(qty) from Dist_Item group by Item_No having sum(qty)>200;

ITEM_	SUM(QTY)
I002	230
I005	450

I001 220

SQL> -- 4. List Dno who delivered more than 50 items for each month

SQL> select dno, sum(qty) as items, to_char(ORDER_DATE, 'MON') as month from Dist_Item group by (dno, to_char(ORDER_DATE, 'MON')) having sum(qty)>200;

DNO	ITEMS	MONTH
D003	320	SEP
D004	220	SEP
D005	340	SEP

SQL> -- 5. Display item details in descending order of price and ascending order of weight

SQL> select * from item order by price DESC, weight;

ITEM_	ITEM_NAME	PRICE	WEIGHT
I004	Oil	120	2
I001	Rice	50	5
I002	Wheat	40	4
I003	Sugar	35	3
I005	Salt	20	1

SQL> -- 6. Show all distributors in alphabetical order of City and DName

SQL> select * from distributor order by DNAME, CITY;

DNO	DNAME	CITY	PHONE
D001	ABC Distributors	Baroda	9876543210
D007	ABCD	Baroda	1234567890
D006	Fast	Pune	9090912345
D005	FastMart	Pune	9090909090
D004	Global Supply	Delhi	9012345678
D003	Om Traders	Mumbai	9988776655
D002	Shree Suppliers	Surat	9123456780

7 rows selected.

```
SQL> -- 7. Calculate average quantity of items distributed on each day.
```

```
SQL> select trunc(order_date) as day, avg(qty) from dist_item group by trunc(order_date);
```

DAY	AVG (QTY)
10-SEP-25	96.6666667
04-SEP-25	80
05-SEP-25	50
06-SEP-25	60
07-SEP-25	200
09-SEP-25	112.5
11-SEP-25	90
12-SEP-25	250
18-SEP-25	120
17-SEP-25	120

```
10 rows selected.
```

```
SQL> -- 8. Find the weight wise average price of items
```

```
SQL> select weight, avg(price) from item group by weight;
```

WEIGHT	AVG (PRICE)
5	50
4	40
3	35
2	85
1	25

```
SQL> -- Question-3 tables: Worker, Job and Assigned;
```

```
SQL> -- 1. Display all workers in descending order of their Wages_Per_Hr
```

```
SQL> select * from Worker order by WAGES_PER_HR desc;
```

ID	NAME	WAGES_PER_HR
----	------	--------------

4	Diana	70.00
2	Bob	60.00
1	Alice	50.00
3	Charlie	45.00

SQL> -- 2. Count workers in each J_Id

SQL> select J_ID, count(W_ID) from Assigned group by J_ID;

J_ID	COUNT(W_ID)
101	2
102	1
103	2

SQL> -- 3. List all the W_Id who has worked for more than 100 hrs

SQL> select W_ID, sum(TOTAL_HRS) as TOTAL_HRS from Assigned group by W_ID having sum(TOTAL_HRS)>100;

W_ID	TOTAL_HRS
1	120
2	110
3	150

SQL> select W_ID, sum(TOTAL_HRS) as TOTAL_HRS from Assigned WHERE status='Completed' group by W_ID having sum(TOTAL_HRS)>100;

W_ID	TOTAL_HRS
1	120
3	150

SQL> -- 4. Count total jobs which are pending till date 5.

SQL> select count(status) as pending_work from Assigned where status='Pending';

PENDING_WORK
2

```
SQL> -- 5. Display all Job_Id according to alphabetical order of their type
SQL> select * from job order by JOB_TYPE;
```

JOB_ID	JOB_TYPE
103	Painting
102	Plumbing
101	Welding

```
SQL> -- day-7.docx file exercise
```

```
SQL> -- Topics: Subqueries
```

- Using sub-query in the FROM clause
- Using co-related sub-queries
- Using multi-column sub-queries
- Using sub-query in an Order-By clause
- Using Exits and Not Exists operator

```
SQL> -- Question-1 tables: STUDENT, Course and Enroll
```

```
SQL> -- 1. Display the name of students who have enrolled for course 'C101'.
```

```
SQL> select fname,lname from student where rollno in(select rollno from enroll where courseno='C101');
```

FNAME	LNAME
Kapil	Sharma
Anjali	Verma

```
SQL> -- 2. Display the name of the courses in which rollno 11 is enrolled.
```

```
SQL> select coursename from course where c_no in(select courseno from enroll where rollno=11);
```

COURSENAME

Mathematics

Physics

```
SQL> -- 3. Display the name of the student who have earned maximum credit.
```

```
SQL> select fname, lname from student where rollno in(select rollno from enroll where earn_credit=(select max(earn_credit) from enroll));
```

FNAME LNAME

Kapil Sharma

Anjali Verma

Ravi Kumar

```
SQL> -- 4. Display details of students who have enrolled in any course.
```

```
SQL> select * from student where rollno in(select rollno from enroll);
```

ROLLNO	FNAME	LNAME	DOB
11	Kapil	Sharma	01-JAN-03
12	Anjali	Verma	21-JUN-02
13	Ravi	Kumar	12-MAR-04

```
SQL> -- 5. Display details of students who have not enrolled in any course.
```

```
SQL> select * from student where rollno not in(select rollno from enroll);
```

ROLLNO	FNAME	LNAME	DOB
14	Pooja	Singh	23-NOV-01

```
SQL> -- 6. Display details of students who have enrolled in all courses.
```

```
SQL> select * from student where rollno in(select rollno from enroll group by rollno having count(distinct courseno)=(select count(*) from course));
```

ROLLNO	FNAME	LNAME	DOB
11	Kapil	Sharma	01-JAN-03

SQL> -- 7. Give the name of the course in which maximum number of students are enrolled.

SQL> **select coursename from course where c_no=(select courseno from enroll group by courseno having count(distinct rollno)=(select max(count(distinct rollno)) from enroll group by courseno));**

COURSENAME

Chemistry

SQL> -- 8. Give the name of the course in which no student has enrolled

SQL> **select coursename from course where c_no not in(select distinct courseno from enroll);**

COURSENAME

RDBMS

SQL> -- 9. Find all students whose credit earned is higher than the average credit of the students in their course. (using correlate sub query)

SQL> **select e.* from enroll e where e.earn_credit > (select avg(e2.earn_credit) from enroll e2 where e2.courseno=e.courseno);**

ROLLNO	COURSENO	EARN_CREDIT
12	C103	4
13	C103	4

SQL> -- 10. List the student detail whose total credit is same as that of `Kapil`

SQL> **select fname from student where rollno in(select rollno from enroll group by rollno having sum(earn_credit) = (select sum(earn_credit) from enroll where rollno=(select rollno from student where fname='Kapil')));**

FNAME

Kapil

Anjali

SQL>

SQL> -- Question-2 tables: Employee, Company and Customer

SQL> -- 1. Find all students whose salary is higher than the average salary of the employees in their departments.

SQL> select fname from employee where emp_no in(select emp_no from company c1 where salary > (select avg(salary) from company c2 where c1.department=c2.department));

FNAME

Jay

Nina

SQL>

SQL> -- 2. List the employee detail whose salary is same as "Jay".

SQL> select * from employee where EMP_NO in(select EMP_NO from company where SALARY = (select salary from company where EMP_NO = (select EMP_NO from employee where FNAME='Jay')));

EMP_NO	FNAME	LNAME	DOB	ADDRESS
105	Meena	Sharma	14-MAR-93	Pune
101	Jay	Shah	01-JAN-90	Mumbai

SQL> -- 3. Display the name of employees who work in 'mrkt' department.

SQL> select fname from employee where emp_no in(select emp_no from company where DEPARTMENT='mrkt');

FNAME

Jay

Amit

SQL> -- 4. Display details of employees not belong to any department.

SQL> select * from employee where emp_no not in(select emp_no from company);

EMP_NO	FNAME	LNAME	DOB	ADDRESS
106	Rahul	Makvana	24-SEP-95	Gujarat

SQL> -- 5. Find out all the customers having same name as the employees. (Using multi column subquery).

SQL> select fname, lname from customer where (fname,lname) in(select fname, lname from employee);

FNAME	LNAME
Jay	Shah
Amit	Patel
Ravi	Kumar

SQL> -- day-8.docx file exercise

SQL> -- Topics: What is Join?

Explain types of Join.

Understand Equi Join with examples

SQL> -- Question-1 tables: Client_master, Product_Master, Salesman_master, SALES_ORDER and Sales_order_Details

SQL> -- 1. Display the salesman name who have processed Order_No 1

SQL> select s.name from SALESMAN_MASTER s join SALES_ORDER o on o.Salesman_No=s.s_no where o.Order_No='0001';

NAME
AMAN

SQL> -- 2. Display datewise (dates in descending order) order details with salesman name and product name.

SQL> select o.ORDER_DATE,s.name as salesman_name,p.DESCRPTION as product_name,od.PRODUCT_RATE from SALES_ORDER o
join SALESMAN_MASTER s on s.S_NO=o.Salesman_No
join SALES_ORDER_DETAILS od on o.Order_No=od.Order_No
join PRODUCT_MASTER p on p.P_NO=od.Product_No
order by o.ORDER_DATE desc;

ORDER_DAT	SALESMAN_NAME	PRODUCT_NAME	PRODUCT_RATE
07-SEP-25	Nidhi	Jacket	3000
06-SEP-25	Sahil	Belt	4000
05-SEP-25	Dev	Shoes	8000
04-SEP-25	Priya	Cap	1800
03-SEP-25	Karan	Kurta	4200
02-SEP-25	Meera	Shirt	5400
01-SEP-25	ASHISH	SAREE	2400
31-AUG-25	RAJ	SKIRT	2800
30-AUG-25	OMKAR	JEANS	2000
29-AUG-25	AMAN	T-SHIRT	3500

10 rows selected.

SQL> -- 3. Display the unique product names, whose order status is still pending

```
SQL> select DISTINCT p.DESCRPTION as product_name from PRODUCT_MASTER p
      join SALES_ORDER_DETAILS od on p.P_NO=od.Product_No
      join SALES_ORDER o on o.Order_No=od.Order_No
      where ORDER_STATUS='Pending';
```

PRODUCT_NAME

 JEANS
 Kurta
 Shoes

SQL> -- 4. Display the Client_name who has made order on 15th Aug, 2025

```
SQL> select c.CLIENT_NAME from CLIENT_MASTER c join SALES_ORDER s on s.Client_No=c.c_no where order_date='15-AUG-25';
```

CLIENT_NAME

 Dhyan Dave

SQL> -- 5. Display the salesman_no and his order_no where quantity ordered and quantity dispatched are not equal

```
SQL> select s.Salesman_No, s.Order_No from Sales_Order s join Sales_Order_Details so on so.Order_No=s.Order_No where so.Qty_Ordered=so.Qty_Dispatched;
```

```
SALE ORDE
---- ----
S001 O001
S003 O003
S005 O005
S009 O009
```

```
SQL> -- Question-2 tables: APPLICANT, ENTRANCE_TEST, ETEST_DETAILS
```

```
SQL> -- 1. Display Entrance Test ID (ETID) Wise highest marks scored by any applicant.
```

```
SQL> SELECT AID, SCORE FROM ETEST_DETAILS e WHERE SCORE = ( SELECT MAX(SCORE) FROM ETEST_DETAILS WHERE ETID = e.ETID);
```

```
AID          SCORE
-----
A002          70
A003          75
A006          90
A009          95
A010          75
```

```
SQL> -- 2. Count ETID wise total number of applicants appeared for the test
```

```
SQL> select etid, count(aid) from ETEST_DETAILS group by etid;
```

```
ETID COUNT(AID)
-----
T001          2
T002          2
T003          2
T004          2
T005          2
```

```
SQL> -- 3. Find the minimum number of applicants in the entrance test.
```

```
SQL> select min(total) from (select etid, count(aid) as total from ETEST_DETAILS group by etid);
```

MIN(TOTAL)

2

SQL> -- 4. Count city wise number of applicants registered

SQL> select city,count(city) from APPLICANT group by city;

CITY	COUNT(CITY)
-----	-----
Baroda	3
Surat	2
Mumbai	1
Delhi	1
Pune	1
Jaipur	1
Chennai	1

SQL> -- 5. Display all the entrance test details for which the applicant "Ohm Patel" appeared

SQL> alter table ETEST_DETAILS drop constraint SYS_C008268;

Table altered.

SQL> select et.* from etest_details et join applicant a on a.aid=et.aid where A_NAME='Ohm Patel';

AID	ETID	ETEST_DAT	SCORE	LOCATION
----	----	-----	-----	-----
A011	T004	31-AUG-25	99	Gujarat

SQL> -- Question-3 tables: Distributor, Item, Dist_Item

SQL> -- 1. Display all the distributor's name who supplies Item_No 5

SQL> select item_no from (select item_no, count(item_no) as total from DIST_ITEM group by item_no) where total=(select max(count(item_no)) as total from DIST_ITEM group by item_no);

ITEM_

I002

I003

SQL> -- 2. Display the item which is distributed maximum time

```
SQL> select distinct i.ITEM_NAME from item i join DIST_ITEM d on d.item_no=i.item_no where i.item_no in(
      select item_no from (select item_no, count(item_no) as total from DIST_ITEM group by item_no)
      where total=(select max(count(item_no)) as total from DIST_ITEM group by item_no));
```

ITEM_NAME

Wheat

Sugar

SQL> -- 3. Display all the items that are distributed by the distributor "Dev Shah"

```
SQL> select d.dname,d.dno,i.item_no,i.item_name from item i join dist_item di on di.item_no=i.item_no join distributor
d on d.dno=di.dno where d.dname='Dev Shah';
```

DNAME	DNO	ITEM_	ITEM_NAME
-------	-----	-------	-----------

Dev Shah	D007	I002	Wheat
----------	------	------	-------

Dev Shah	D007	I003	Sugar
----------	------	------	-------

SQL> -- 4. Display the Item_Name and Quantity that are received in month of July in 2021

```
SQL> select i.item_name,di.qty,di.order_date from item i join dist_item di on di.item_no=i.item_no where di.order_date
like '%JUL-21';
```

ITEM_NAME	QTY	ORDER_DAT
-----------	-----	-----------

Rice	100	24-JUL-21
------	-----	-----------

Sugar	100	24-JUL-21
-------	-----	-----------

Salt	200	24-JUL-21
------	-----	-----------

Salt	250	24-JUL-21
------	-----	-----------

SQL> -- 5. Display all the items whose price is less than 1000 and received Qty =10

```
SQL> select distinct i.item_name from item i join dist_item di on di.item_no=i.item_no where i.price<1000 AND
di.qty=10;
```

ITEM_NAME

Sugar
Salt

SQL> -- Question-4 tables: Worker, Job and Assigned;

SQL> -- 1. Display all the workers assigned for Job "Assemble"

SQL> select w.name,j.job_type from job j join assigned a on a.j_id=j.job_id join worker w on w.id=a.w_id where
j.job_type='Assemble';

NAME	JOB_TYPE
-----	-----
Charlie	Assemble
Bob	Assemble

SQL> -- 2. Display total hours of the worker "Deep Jha" spent on Job "Packing"

SQL> select w.name,a.total_hrs from assigned a join worker w on w.id=a.w_id join job j on j.job_id=a.j_id where
w.name='Deep Jha' AND j.job_type='Packing';

NAME	TOTAL_HRS
-----	-----
Deep Jha	80

SQL> -- 3. Display all the job types that are starting from Sept-2021.

SQL> select j.* from job j join assigned a on a.j_id=j.job_id where start_date like '%SEP-21';

JOB_ID	JOB_TYPE
-----	-----
101	Cutting
103	Assemble

SQL> -- 4. What are the total hours allocated for Job "Cutting"

SQL> select sum(a.total_hrs) from job j join assigned a on a.j_id=j.job_id where j.JOB_TYPE='Cutting' group by a.j_id;

SUM(A.TOTAL_HRS)

SQL> -- 5. Display all the jobs that are still pending

SQL> **select j.job_type from job j join assigned a on a.j_id=j.job_id where a.status='Pending';**

```
JOB_TYPE
-----
Packing
Cutting
```

SQL> commit;

Commit complete.

◆ What is a JOIN?

A **JOIN** is used to **combine rows from two or more tables** based on a **related column** between them.

👉 JOINS are used to **retrieve meaningful data** from multiple tables.

◆ Why JOIN is required?

- Data is stored in **multiple related tables**
- JOIN helps to **avoid data redundancy**
- JOIN helps to **fetch combined information**

📌 Example: Employee details + Department details

◆ Types of JOIN

1 INNER JOIN

Returns **only matching rows** from both tables.

```
SELECT * FROM emp e INNER JOIN dept d ON e.deptno = d.deptno;
```

✓ Only common records

2 LEFT OUTER JOIN

Returns **all rows from left table** and matching rows from right table.

```
SELECT * FROM emp e LEFT JOIN dept d ON e.deptno = d.deptno;
```

✓ Unmatched rows from right table show **NULL**

3 RIGHT OUTER JOIN

Returns **all rows from right table** and matching rows from left table.

```
SELECT * FROM emp e RIGHT JOIN dept d ON e.deptno = d.deptno;
```

4 FULL OUTER JOIN

Returns **all rows from both tables**.

```
SELECT * FROM emp e FULL JOIN dept d ON e.deptno = d.deptno;
```

5 CROSS JOIN

Returns **Cartesian product** (all combinations).

```
SELECT * FROM emp CROSS JOIN dept;
```

⚠ Use carefully

◆ Equi Join (IMPORTANT)

◆ What is Equi Join?

An **Equi Join** is a join that uses the **equality operator (=)** to match rows between tables.

👉 It is the **most commonly used join**.

◆ Equi Join Example (Old Syntax)

```
SELECT e.empno, e.ename, d.dname FROM emp e, dept d WHERE e.deptno = d.deptno;
```

◆ Equi Join Example (ANSI Syntax)

```
SELECT e.empno, e.ename, d.dname FROM emp e JOIN dept d ON e.deptno = d.deptno;
```

✓ Both give the **same result**

◆ Difference: Equi Join vs Non-Equi Join

Equi Join	Non-Equi Join
Uses =	Uses <, >, BETWEEN
Exact match	Range-based match
Most common	Less common

◆ Non-Equi Join Example

```
SELECT e.ename, s.grade FROM emp e, salgrade s WHERE e.sal BETWEEN s.losal AND s.hisal;
```

★ Viva-Ready Short Answer (MEMORIZE)

A join is used to combine data from two or more tables using a related column. Types of joins include inner join, outer join, cross join, and equi join. An equi join uses the equality operator to match rows between tables.

🧠 One-Line Exam Answers

- JOIN → combines tables
- INNER JOIN → matching rows only
- OUTER JOIN → includes unmatched rows
- EQUI JOIN → uses = operator

```
SQL> -- day-9.docx file exercise
```

```
SQL> -- Topics: Set Operation: UNION, UNION ALL, INTERSECT, MINUS
```

```
SQL> -- Question-1 tables: Worker, Job, Assigned
```

```
SQL> -- 1. Display all the workers' Id assigned for Job 1 and 2 both (J_Id =1 and J_Id=2)
```

```
SQL> select w.* from worker w join Assigned a on a.W_ID=w.ID where a.J_ID in(101, 102);
```

ID	NAME	WAGES_PER_HR
1	Alice	50
2	Deep Jha	60
4	Diana	70

```
SQL> -- 2. List the workers who are working for J_Id =1 and not for J_Id = 2
```

```
SQL> select w.* from Worker w where w.id in(select a.w_id from Assigned a where a.j_id=101) AND w.id not in(select a.w_id from Assigned a where a.j_id=102);
```

ID	NAME	WAGES_PER_HR
1	Alice	50
4	Diana	70

```
SQL> -- 3. Display the workers Id who are working for more than 6 hours and status is pending. And Union them
SQL> select distinct w.id from Worker w join Assigned a on w.id=a.w_id where a.total_hrs>6 AND a.status='Pending'
UNION select distinct w.id from Worker w join Assigned a on w.id=a.w_id where a.total_hrs<6 AND a.status='Completed';
```

ID
2
4
3

```
SQL> -- Question-2 tables: FYRankers, SYRankers
```

```
SQL> -- 1. Display the name of the student who is ranker in 'FY' or 'SY'
SQL> select name from FYRankers UNION select name from SYRankers;
```

NAME
Anna White
Chris Lee
Emily Davis
Jane Smith
John Doe
Michael Brown

6 rows selected.

```
SQL> -- 2. Display the name of the student who is ranker in 'FY' or 'SY' including duplicate data
SQL> select name from FYRankers UNION ALL select name from SYRankers;
```

NAME

```
John Doe
Jane Smith
Emily Davis
Michael Brown
Emily Davis
Michael Brown
Chris Lee
Anna White
```

8 rows selected.

```
SQL> -- 3. Display the name of the students who is ranker in 'FY' or 'SY' and having SPI more than 7
SQL> select name from FYRankers where spi>7 UNION select name from SYRankers where spi>7;
```

```
NAME
-----
Chris Lee
Emily Davis
Jane Smith
John Doe
Michael Brown
```

```
SQL> -- 4. Display the name of the student who is ranker in both FY and SY
SQL> select name from FYRankers INTERSECT select name from SYRankers;
```

```
NAME
-----
Emily Davis
Michael Brown
```

```
SQL> -- 5. Display the name of the student who is ranker in FY but not in SY
SQL> select name from FYRankers MINUS select name from SYRankers;
```

```
NAME
-----
Jane Smith
John Doe
```

```
SQL> -- 6. Display the name of the student who is ranker in SY but not in FY
```

```
SQL> select name from SYRankers MINUS select name from FYRankers;
```

```
NAME
```

```
-----
```

```
Anna White
```

```
Chris Lee
```

```
SQL> -- Question-3 tables: customer, supplier
```

```
SQL> -- 1. Display distinct city of both the customer and supplier table in sorted order.
```

```
SQL> select city from customer UNION select city from supplier;
```

```
CITY
```

```
-----
```

```
Ahmedabad
```

```
London
```

```
New York
```

```
SQL> -- 2. Display city of both the tables with duplicate data.
```

```
SQL> select city from customer UNION ALL select city from supplier;
```

```
CITY
```

```
-----
```

```
Ahmedabad
```

```
New York
```

```
London
```

```
Ahmedabad
```

```
Ahmedabad
```

```
New York
```

```
London
```

```
Ahmedabad
```

```
8 rows selected.
```

```
SQL> -- 3. Display the details of Ahmedabad city of both customer and supplier table.
```

```
SQL> select name,city from customer where city='Ahmedabad' UNION select name,city from supplier where city='Ahmedabad';
```

NAME	CITY
-----	-----
Charlie Black	Ahmedabad
John Doe	Ahmedabad
Sup1	Ahmedabad
Sup4	Ahmedabad

```
SQL> -- 4. Display the name of students who have enrolled for `html` course.
```

```
SQL> select s.student_name from students1 s join enrollment1 e on e.student_id=s.student_id where e.course_name='HTML';
```

STUDENT_NAME

Eve Adams

```
SQL> -- 5. Display customer details who are living in `ahmedabad` and name start with `A`.
```

```
SQL> select * from customers where city='Ahmedabad' AND name like 'A%';
```

CUST_ID	NAME	CITY	COUNTRY
-----	-----	-----	-----
5	Avinav josi	Ahmedabad	India

```
SQL> -- 6. Find the details of the supplier who are not customer.
```

1 UNION

- Combines results of two SELECT statements
- **Removes duplicate rows**

```
SELECT deptno FROM emp UNION SELECT deptno FROM dept;
```

✓ No duplicates

✓ Sorted result (by default)

2 UNION ALL

- Combines results

- **Keeps duplicate rows**

```
SELECT deptno FROM emp UNION ALL SELECT deptno FROM dept;
```

- ✓ Faster than UNION
 - ✓ Duplicates allowed
-

3 INTERSECT

- Returns **common rows** from both SELECT statements.

```
SELECT deptno FROM emp INTERSECT SELECT deptno FROM dept;
```

- ✓ Only matching values
-

4 MINUS

- Returns rows from **first SELECT not present in second**

```
SELECT deptno FROM emp MINUS SELECT deptno FROM dept;
```

- ✓ First - Second
 - ✓ Order matters
-

◆ Simple Example (Easy to remember)

Table A → {1, 2, 3}

Table B → {3, 4, 5}

Operator	Result
UNION	1, 2, 3, 4, 5
UNION ALL	1, 2, 3, 3, 4, 5
INTERSECT	3
MINUS	1, 2

◆ Difference: UNION vs UNION ALL

UNION	UNION ALL
Removes duplicates	Keeps duplicates
Slower	Faster
Sorting done	No sorting

◆ Difference: JOIN vs SET OPERATOR

JOIN	SET Operator
Combines columns	Combines rows
Works on tables	Works on result sets
Horizontal merge	Vertical merge

★ Viva-Ready Short Answer (MEMORIZE)

Set operators are used to combine the results of two or more SELECT statements. The main set operators are UNION, UNION ALL, INTERSECT, and MINUS.

🧠 One-Line Exam Answers

- UNION → combines results, removes duplicates
- UNION ALL → combines results, keeps duplicates
- INTERSECT → common records
- MINUS → records from first query not in second

```
SQL> -- day-10.docx file exercise
```

```
SQL> -- Topics: Transaction control commands - Commit, Rollback, Savepoint
```

```
SQL> -- create savepoint sp1
```

```
SQL> savepoint sp1;
```

Savepoint created.

```
SQL> update PRODUCT set price=100 where prod_id=1;
```

1 row updated.

```
SQL> savepoint sp2;
```

Savepoint created.

```
SQL> update PRODUCT set price=200 where prod_id=2;
```

1 row updated.

```
SQL> select * from PRODUCT;
```

PROD_ID	PROD_NAME	SUPPLIER_ID	CAT_ID	UNIT	PRICE
1	Product 1	101	1	Piece	100
2	Product 2	102	2	Box	200
3	Product 3	103	3	Pack	30
4	Product 4	104	1	Bag	15

```
SQL> rollback to savepoint sp1;
```

Rollback complete.

```
SQL> select * from PRODUCT;
```

PROD_ID	PROD_NAME	SUPPLIER_ID	CAT_ID	UNIT	PRICE
1	Product 1	101	1	Piece	10
2	Product 2	102	2	Box	20
3	Product 3	103	3	Pack	30
4	Product 4	104	1	Bag	15

```
SQL> savepoint sp1;
```

Savepoint created.

```
SQL> update PRODUCT set price=100 where prod_id=1;
```

1 row updated.

SQL> **savepoint sp2;**

Savepoint created.

SQL> update PRODUCT set price=200 where prod_id=2;

1 row updated.

SQL> select * from PRODUCT;

PROD_ID	PROD_NAME	SUPPLIER_ID	CAT_ID	UNIT	PRICE
1	Product 1	101	1	Piece	100
2	Product 2	102	2	Box	200
3	Product 3	103	3	Pack	30
4	Product 4	104	1	Bag	15

SQL> **rollback to savepoint sp2;**

Rollback complete.

SQL> select * from PRODUCT;

PROD_ID	PROD_NAME	SUPPLIER_ID	CAT_ID	UNIT	PRICE
1	Product 1	101	1	Piece	100
2	Product 2	102	2	Box	20
3	Product 3	103	3	Pack	30
4	Product 4	104	1	Bag	15

SQL> **COMMIT;**

Commit complete.

SQL> **-- Question-1 tables: products, order_details**

SQL> set serveroutput on;

```
SQL> -- 1. Write a PLSQL block to display total number of products ordered in Order_ID = 3
SQL> declare
  2     total_products number;
  3 begin
  4     select count(prod_id) INTO total_products from ORDER_DETAILS where ORDER_ID=3;
  5     dbms_output.put_line('total number of products: ' || total_products);
  6* end;
SQL> /
total number of products: 3
```

PL/SQL procedure successfully completed.

```
SQL> -- 2. Write a PLSQL block to update the price (actual price + 5) of product with Id = 2
SQL> begin
  2     update PRODUCTS set PRICE=PRICE+5 where PROD_ID=2;
  3* end;
SQL> /
```

PL/SQL procedure successfully completed.

```
SQL> -- 3. Write a PLSQL block to delete the products of Cat_Id = 3
SQL> begin
  2     delete from PRODUCTS where Cat_Id=3;
  3* end;
SQL> /
```

PL/SQL procedure successfully completed.

```
SQL> -- 4. Write a PLSQL block to insert any product whose cat_id = 3
SQL>
  1 begin
  2     INSERT INTO PRODUCTS (Prod_ID, Prod_Name, Supplier_ID, Cat_ID, Unit, Price) VALUES (3, 'Product 3', 103, 3,
'Pack', 30);
  3* end;
SQL> /
```

PL/SQL procedure successfully completed.

```
SQL> select * from PRODUCTS;
```

PROD_ID	PROD_NAME	SUPPLIER_ID	CAT_ID	UNIT	PRICE
3	Product 3	103	3	Pack	30
1	Product 1	101	1	Piece	100
2	Product 2	102	2	Box	30
4	Product 4	104	1	Bag	15

```
SQL> -- 5. Write a PLSQL block to display Supplier_Id and their total number of products they supply
```

```
SQL> BEGIN
```

```
2   dbms_output.put_line('SUPPLIER_ID          TOTAL_PRODUCTS');
3   dbms_output.put_line('-----');
4   for data in (select Supplier_ID, count(Prod_ID) as total_products from PRODUCT group by Supplier_ID)
5   loop
6       dbms_output.put_line(' ' || data.Supplier_ID || ' ' || data.total_products);
7   end loop;
8   END;
9   /
```

SUPPLIER_ID	TOTAL_PRODUCTS
101	1
102	1
103	1
104	1

```
PL/SQL procedure successfully completed.
```

```
SQL> -- day-11.docx file exercise
```

```
SQL> -- Topics: Introduction to PL/SQL:
```

Advantages of PL/SQL
The Generic PL/SQL Block,
PL/SQL Execution Environment,
Control Structure
• Conditional Control

SQL> -- Question-1 if-else and Exception

SQL> set serveroutput on;

SQL> -- 1. Print a static string "Hello Every One...!" using an anonymous PLSQL block and execute it.

```
SQL> BEGIN
  2     DBMS_OUTPUT.PUT_LINE('Hello Every One...!');
  3 END;
  4 /
```

Hello Every One...!

PL/SQL procedure successfully completed.

SQL> -- 2. Write a PLSQL block to display a greeting message like: "Hi!! Today is 3rd November 2021, Friday.

```
SQL> BEGIN
  2     dbms_output.put_line('Hey!! Today is ' || to_char(sysdate, 'DD Month YYYY Day'));
  3 END;
  4 /
```

Hey!! Today is 13 October 2025 Monday

PL/SQL procedure successfully completed.

SQL> -- 3. Declare a string variable to store student's name, define three integer variables to store marks of 3 subjects (out of 50) of that student. Write a PLSQL code to calculate total of all three subjects and print the result in percentage.

```
SQL> declare
  2     name varchar(15);
  3     s1 number;
  4     s2 number;
  5     s3 number;
  6     total number;
  7     per number;
```



```

7      b:=&b;
8      ans:= a/b;
9      dbms_output.put_line('Ans is ' || ans);
10 Exception
11      when ZERO_DIVIDE then
12          dbms_output.put_line('B value is 0');
13 end;
14 /

```

Enter value for a: 11

old 6: a:=&a;

new 6: a:=11;

Enter value for b: 0

old 7: b:=&b;

new 7: b:=0;

B value is 0

PL/SQL procedure successfully completed.

SQL> -- 5. Write a user defined exception for above program 3 where if marks are less than 0 then appropriate error message must be shown as exception.

SQL> declare

```

2      name varchar(15);
3      s1 number;
4      s2 number;
5      s3 number;
6      total number;
7      per number;
8      marksNotZero EXCEPTION;
9  begin
10     name:='&name';
11     s1:=&s1;
12     s2:=&s2;
13     s3:=&s3;
14     if s1=0 OR s2=0 OR s3=0 then
15         RAISE marksNotZero;
16     end if;
17     total := s1+s2+s3;

```

```

18     per := total/3;
19     dbms_output.put_line('Student name: ' || name);
20     dbms_output.put_line('Total marks: ' || total);
21     dbms_output.put_line('Percentage: ' || per);
22 Exception
23     when marksNotZero then
24         dbms_output.put_line('Marks less than zero! Negative marks not allowed');
25 end;
26 /

```

Enter value for name: Vishal

```

old 10:      name:='&name';
new 10:      name:='Vishal';

```

Enter value for s1: 0

```

old 11:      s1:='&s1';
new 11:      s1:=0;

```

Enter value for s2: 13

```

old 12:      s2:='&s2';
new 12:      s2:=13;

```

Enter value for s3: 14

```

old 13:      s3:='&s3';
new 13:      s3:=14;

```

Marks less than zero! Negative marks not allowed

PL/SQL procedure successfully completed.

SQL> -- 6. Write a PLSQL block to find the largest of three numbers

SQL> declare

```

2     no1 number;
3     no2 number;
4     no3 number;
5 begin
6     no1:=&no1;
7     no2:=&no2;
8     no3:=&no3;
9     if no1>no2 AND no1>no3 then
10         dbms_output.put_line('max number is: ' || no1);
11     elsif no2>no3 then

```

```

12         dbms_output.put_line('max number is: ' || no2);
13     else
14         dbms_output.put_line('max number is: ' || no3);
15     end if;
16 end;
17 /

```

Enter value for no1: 12

old 6: no1:=&no1;

new 6: no1:=12;

Enter value for no2: 13

old 7: no2:=&no2;

new 7: no2:=13;

Enter value for no3: 14

old 8: no3:=&no3;

new 8: no3:=14;

max number is: 14

PL/SQL procedure successfully completed.

◆ Introduction to PL/SQL

PL/SQL (Procedural Language / SQL) is Oracle's **procedural extension of SQL**.

It allows us to write **blocks of code** that include SQL statements along with **procedural features** like loops, conditions, variables, and exceptions.

👉 SQL = *what to do*

👉 PL/SQL = *how to do*

◆ Advantages of PL/SQL

1 Block structure

- Code is written in blocks → easy to read & maintain

2 Reduced network traffic

- Multiple SQL statements executed together

3 Better performance

- SQL + logic executed at database level

4 Supports programming features

- Variables
- Loops
- Conditions
- Exception handling

5 Security

- Logic stored inside database
- Users don't see internal queries

◆ The Generic PL/SQL Block

Structure of a PL/SQL block

```
DECLARE
    -- Declaration section (variables, cursors)
BEGIN
    -- Executable statements
EXCEPTION
    -- Error handling
END;
/
```

Sections explained:

Section	Purpose
DECLARE	Declare variables, cursors
BEGIN	Main executable code
EXCEPTION	Handle runtime errors
END	End of block

📌 **DECLARE and EXCEPTION are optional,**

📌 **BEGIN-END is mandatory**

◆ PL/SQL Execution Environment

PL/SQL code can be executed in:

- 1 SQL*Plus / SQL Developer
- 2 Stored Procedures / Functions
- 3 Triggers
- 4 Packages

How execution works:

- PL/SQL engine processes procedural code
- SQL engine executes SQL statements
- Both engines communicate internally

👉 This improves **speed and efficiency**

◆ **Control Structures in PL/SQL**

Control structures allow us to **control the flow of execution**.

There are **3 types**:

1. Conditional Control
2. Iterative Control
3. Sequential Control

(Your syllabus asks **Conditional Control**, so explained below 👉)

◆ **Conditional Control Structures**

Used to **make decisions** in PL/SQL.

◆ **1 IF Statement**

```
IF condition THEN
    statements;
END IF;
```

📌 Executes when condition is TRUE

◆ **2 IF-ELSE Statement**

```
IF condition THEN
    statements;
ELSE
    statements;
END IF;
```

📌 Executes one block based on condition

◆ **3 IF-ELSIF-ELSE Statement**

```
IF condition1 THEN
    statements;
ELSIF condition2 THEN
    statements;
```

```
ELSE
    statements;
END IF;
```

📌 Used for **multiple conditions**

◆ **Example (Simple)**

```
DECLARE
    sal NUMBER := 30000;
BEGIN
    IF sal > 25000 THEN
        DBMS_OUTPUT.PUT_LINE('High Salary');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Low Salary');
    END IF;
END;
/
```

★ **Viva-Ready Short Answer (MEMORIZE)**

PL/SQL is Oracle's procedural extension of SQL that supports variables, conditions, loops, and exception handling. It improves performance, security, and maintainability. A PL/SQL block consists of DECLARE, BEGIN, EXCEPTION, and END sections. Conditional control structures like IF, IF-ELSE, and IF-ELSIF are used to control program execution based on conditions.

🧠 **One-Line Viva Answers**

- PL/SQL → procedural extension of SQL
- Advantage → better performance & control
- Block structure → DECLARE, BEGIN, EXCEPTION, END
- Conditional control → IF, IF-ELSE, IF-ELSIF

```
SQL> -- day-12.docx file exercise
```

```
SQL> -- PL/SQL Control Structure (Cont...)
```

```
SQL> --      •      Iterative Control
```

```
SQL> --      •      Sequential Control
```

```
SQL> set serveroutput on;
```

```
SQL> -- 1. Write a PLSQL block to print all the prime numbers between 1 to 50
```

```
SQL> ed
```

```
Wrote file afiedt.buf
```

```
1  declare
2      i number;
3      j number;
4      flag boolean;
5  begin
6      for i IN 1..50 loop
7          flag:= true;
8          for j IN 2..trunc(sqrt(i)) loop
9              if mod(i,j)=0 then
10                 flag:= false;
11             end if;
12         end loop;
13         if flag then
14             dbms_output.put_line(i);
15         end if;
16     end loop;
17* end;
```

```
SQL> /
```

```
1
2
3
5
7
11
13
```

17
19
23
29
31
37
41
43
47

PL/SQL procedure successfully completed.

SQL> -- 2. Display all the integer numbers between 4 to 40 which are divisible by 3 using "Exit When" statement.

SQL> DECLARE

```
2         i number := 4;
3 BEGIN
4     LOOP
5         EXIT WHEN i > 40;
6         if mod(i,3)=0 then
7             dbms_output.put_line(i);
8         end if;
9         i:=i+1;
10    END LOOP;
11 END;
12 /
```

6
9
12
15
18
21
24
27
30
33
36
39

PL/SQL procedure successfully completed.

SQL> -- 3. Implement a PL/SQL Block which takes input number of rows and displays triangle.

SQL> -- *

SQL> -- **

SQL> -- ***

SQL> -- ****

SQL> ed

Wrote file afiedt.buf

```
1  declare
2      rows number;
3      i number;
4      j number;
5  begin
6      rows:=&rows;
7      for i in 1..rows loop
8          for j in 1..i loop
9              dbms_output.put('*');
10             end loop;
11             dbms_output.put_line(' ');
12         end loop;
13* end;
```

SQL> /

Enter value for rows: 5

old 6: rows:=&rows;

new 6: rows:=5;

*

**

PL/SQL procedure successfully completed.

SQL> -- 4. Implement PL/SQL block that inputs a string and checks palindrome or not.

```

SQL> -- Note: String is Palindrome if reverse of string equals to the original string.
SQL> -- Eg. Input String = "aibohphobia"
SQL> -- Reverse of string = "aibohphobia"
SQL> -- Output: "aibohphobia" is Palindrome.
SQL> --
SQL> -- Eg Input String = "level"
SQL> -- Reverse of string = "level"
SQL> -- Output: "level" is Palindrome"

```

```

SQL> ed
Wrote file afiedt.buf

```

```

1  declare
2      str varchar(20);
3      str_rev varchar(20);
4  begin
5      dbms_output.put('    Enter String: ');
6      str:= '&str';
7      for i IN REVERSE 1..length(str) loop
8          str_rev := str_rev || substr(str,i,1);
9      end loop;
10     dbms_output.put_line('    Input String : ' || str);
11     dbms_output.put_line('    Reverse String: ' || str_rev);
12     if str = str_rev then
13         dbms_output.put_line('    Output: "' || str || '" is palindrome string');
14     else
15         dbms_output.put_line('    Output: "' || str || '" is not palindrome string');
16     end if;
17* end;

```

```
SQL> /
```

```

Enter value for str: vishal
old   6:      str:= '&str';
new   6:      str:= 'vishal';
Enter String:    Input String : vishal
Reverse String: lahsiv
Output: "vishal" is not palindrome string

```

PL/SQL procedure successfully completed.

SQL> /

Enter value for str: level

old 6: str:= '&str';

new 6: str:= 'level';

Enter String: Input String : level

Reverse String: level

Output: "level" is palindrome string

PL/SQL procedure successfully completed.

SQL> -- **day-13.docx** file exercise

SQL> -- **Topics:- Understanding Cursor, Cursor FOR loops Parameterized Cursor, Cursor within Cursor**

SQL> -- **(I)Create a table Student (R_No, Name, Sub1, Sub2, Sub3, Total, Grade)**

SQL> -- **Insert 10 records in the above table where each subject carries maximum 100 marks. Don't enter the total and grade manually.**

SQL> CREATE TABLE Student1 (

2 R_No NUMBER PRIMARY KEY,

3 Name VARCHAR2(12),

4 Sub1 NUMBER(3),

5 Sub2 NUMBER(3),

6 Sub3 NUMBER(3),

7 Total NUMBER(5),

8 Grade VARCHAR2(5)

9);

Table created.

```
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (1, 'Amit', 78, 65, 80);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (2, 'Nisha', 90, 88, 92);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (3, 'Rahul', 55, 60, 58);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (4, 'Priya', 40, 45, 42);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (5, 'Karan', 95, 90, 85);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (6, 'Sneha', 72, 68, 70);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (7, 'Ravi', 30, 25, 35);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (8, 'Meena', 65, 70, 60);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (9, 'Arjun', 85, 80, 78);
1 row created.
SQL> INSERT INTO Student1 (R_No, Name, Sub1, Sub2, Sub3) VALUES (10, 'Pooja', 50, 55, 53);
1 row created.
```

```
SQL> COMMIT;
```

Commit complete.

```
SQL> select * from student1;
```

R_NO	NAME	SUB1	SUB2	SUB3	TOTAL	GRADE
1	Amit	78	65	80		
2	Nisha	90	88	92		
3	Rahul	55	60	58		
4	Priya	40	45	42		
5	Karan	95	90	85		
6	Sneha	72	68	70		

7	Ravi	30	25	35
8	Meena	65	70	60
9	Arjun	85	80	78
10	Pooja	50	55	53

10 rows selected.

SQL> -- 1. Write a PLSQL block to calculate and update the Total for each and every student.

SQL> ed

Wrote file afiedt.buf

```

1  begin
2      UPDATE student1 set total=(sub1+sub2+sub3);
3* end;
SQL> /

```

PL/SQL procedure successfully completed.

SQL> set serveroutput on;

SQL> select * from student1;

R_NO	NAME	SUB1	SUB2	SUB3	TOTAL	GRADE
1	Amit	78	65	80	223	
2	Nisha	90	88	92	270	
3	Rahul	55	60	58	173	
4	Priya	40	45	42	127	
5	Karan	95	90	85	270	
6	Sneha	72	68	70	210	
7	Ravi	30	25	35	90	
8	Meena	65	70	60	195	
9	Arjun	85	80	78	243	
10	Pooja	50	55	53	158	

10 rows selected.

SQL> -- 2. Calculate the grade of all students, based to total (>70 AA, >60 A, >50 B, >35 C, else Fail)

SQL> ed

Wrote file afiedt.buf

```
1  begin
2      update student1 set grade=
3      case
4          when total>270 then 'AA'
5          when total>250 then 'A'
6          when total>200 then 'B'
7          when total>150 then 'C'
8          else 'Fail'
9      end;
10* end;
SQL> /
```

PL/SQL procedure successfully completed.

SQL> select * from student1;

R_NO	NAME	SUB1	SUB2	SUB3	TOTAL	GRADE
1	Amit	78	65	80	223	B
2	Nisha	90	88	92	270	A
3	Rahul	55	60	58	173	C
4	Priya	40	45	42	127	Fail
5	Karan	95	90	85	270	A
6	Sneha	72	68	70	210	B
7	Ravi	30	25	35	90	Fail
8	Meena	65	70	60	195	C
9	Arjun	85	80	78	243	B
10	Pooja	50	55	53	158	C

10 rows selected.

SQL> -- 3. Write a Cursor to find the first 3 rankers based on the total marks.

SQL> ed

Wrote file afiedt.buf

```

1  DECLARE
2      nm Student1.Name%TYPE;
3      t Student1.Total%TYPE;
4      g Student1.Grade%TYPE;
5      count_n NUMBER := 0;
6      CURSOR cur IS SELECT Name, Total, Grade FROM Student1 ORDER BY Total DESC;
7  BEGIN
8      OPEN cur;
9      LOOP
10         FETCH cur INTO nm, t, g;
11         EXIT WHEN count_n = 3 OR cur%NOTFOUND;
12         count_n := count_n + 1;
13         DBMS_OUTPUT.PUT_LINE(nm || ' ' || t || ' ' || g);
14     END LOOP;
15     CLOSE cur;
16* END;

```

SQL> /

Karan 270 A

Nisha 270 A

Arjun 243 B

PL/SQL procedure successfully completed.

SQL> -- (II)

SQL> -- 1. Add a Salary and Bonus column in the Faculty_Master Table and calculate the bonus of each faculty of "MCA" department which is based on the 5% of their salary. If the salary is less than 25000, then raise the exception.

SQL> ALTER TABLE Faculty_Master ADD (Department VARCHAR2(10), Salary NUMBER, Bonus NUMBER);

Table altered.

SQL> UPDATE Faculty_Master SET Department = 'MCA', Salary = 30000 WHERE Faculty_ID = 1;

1 row updated.

SQL> UPDATE Faculty_Master SET Department = 'MCA', Salary = 24000 WHERE Faculty_ID = 2;

1 row updated.

SQL> UPDATE Faculty_Master SET Department = 'MBA', Salary = 40000 WHERE Faculty_ID = 3;

1 row updated.

SQL> UPDATE Faculty_Master SET Department = 'MCA', Salary = 35000 WHERE Faculty_ID = 4;

1 row updated.

SQL> select * from faculty_master;

FACULTY_ID	FACULTY_NAME	DOJ	DEPARTMENT	SALARY	BONUS
1	Dr. Smith	15-JUN-10	MCA	30000	
2	Prof. Johnson	01-SEP-12	MCA	24000	
3	Dr. Lee	20-JAN-15	MBA	40000	
4	Prof. Brown	05-NOV-18	MCA	35000	

```
SQL> ed
Wrote file afiedt.buf

 1  declare
 2      fid Faculty_Master.faculty_id%type;
 3      fnm Faculty_Master.faculty_name%type;
 4      dep Faculty_Master.department%type;
 5      sal Faculty_Master.salary%type;
 6      cursor cur is select FACULTY_ID, FACULTY_NAME, DEPARTMENT, SALARY from Faculty_Master where DEPARTMENT =
'MCA';
 7      low_salary EXCEPTION;
 8  begin
 9      open cur;
10      loop
11          fetch cur into fid, fnm, dep, sal;
12          exit when cur%NOTFOUND;
13          begin
14              if sal < 25000 then
15                  raise low_salary;
16              end if;
17              update Faculty_Master set BONUS = sal * 0.05 where FACULTY_ID = fid;
18          exception
19              when low_salary then
20                  dbms_output.put_line('Exception: Salary < 25000 for Faculty ID: ' || fid || ' Name:
' || fnm);
21          end;
22      end loop;
23      close cur;
24* end;
SQL> /
Exception: Salary < 25000 for Faculty ID: 2 Name: Prof. Johnson
```

PL/SQL procedure successfully completed.

```
SQL> select * from Faculty_Master;
```

FACULTY_ID	FACULTY_NAME	DOJ	DEPARTMENT	SALARY	BONUS
------------	--------------	-----	------------	--------	-------

1	Dr. Smith	15-JUN-10	MCA	30000	1500
2	Prof. Johnson	01-SEP-12	MCA	24000	
3	Dr. Lee	20-JAN-15	MBA	40000	
4	Prof. Brown	05-NOV-18	MCA	35000	1750

SQL>

SQL> -- 2. Display name of 2 faculties getting maximum bonus.

SQL> select * from Faculty_Master;

FACULTY_ID	FACULTY_NAME	DOJ	DEPARTMENT	SALARY	BONUS
1	Dr. Smith	15-JUN-10	MCA	30000	1500
2	Prof. Johnson	01-SEP-12	MCA	24000	1200
3	Dr. Lee	20-JAN-15	MBA	40000	2000
4	Prof. Brown	05-NOV-18	MCA	35000	1750

SQL> declare

2 fid Faculty_Master.faculty_id%type;

3 fnm Faculty_Master.faculty_name%type;

4 bon Faculty_Master.bonus%type;

5 num number := 0;

6 cursor cur is select FACULTY_ID, FACULTY_NAME, BONUS from Faculty_Master order by BONUS desc;

7 begin

8 dbms_output.put_line('The 2 faculties who got the maximum bonus are: ');

9 dbms_output.put_line('ID Name');

10 open cur;

11 loop

12 fetch cur into fid, fnm, bon;

13 exit when num = 2 OR cur%NOTFOUND;

14 num := num + 1;

15 dbms_output.put_line(fid || ' ' || fnm);

16 end loop;

17 close cur;

18* end;

SQL> /

The 2 faculties who got the maximum bonus are:

ID Name

3 Dr. Lee
4 Prof. Brown

PL/SQL procedure successfully completed.

SQL> -- (III)

SQL> -- Use following tables are create a report as shown below.

supplier (sid, sname, contactnum), parts (pid, pname, color, unitrate), catalog (sid, pid, qty), [primary
key(sid,pid)]

Write a PLSQL Cursor to take a supplier's name as an input from the user and prepare a report in the following format:

PART ID	PART NAME	QUANTITY	UNIT PRICE	TOTAL
---------	-----------	----------	------------	-------

Grand Total:

Raise a user defined exception when the supplier's name not found in the database table.

SQL> ed

Wrote file afiedt.buf

```
1  DECLARE
2      p_name VARCHAR2(20);
3      -- supplier
4      s_sid suppliers.s_id%TYPE;
5      s_snm suppliers.s_name%TYPE;
6      -- catalog
7      c_pid catalog.part_id%TYPE;
8      c_qty catalog.qty%TYPE;
9      -- parts
10     p_pid parts.part_id%TYPE;
11     p_pnm parts.part_name%TYPE;
12     p_rate parts.unit_rate%TYPE;
13     total NUMBER;
14     grand_total NUMBER := 0;
15     supplier_not_found EXCEPTION;
16     -- supplier cursor
17     CURSOR cur_supplier IS SELECT s_id, s_name FROM suppliers WHERE s_name = p_name;
```

```

18  -- catalog cursor (depends on supplier)
19  CURSOR cur_catalog IS SELECT part_id, qty FROM catalog WHERE s_id = s_sid;
20  -- parts cursor
21  CURSOR cur_parts IS SELECT part_id, part_name, unit_rate FROM parts;
22  BEGIN
23  -- take input
24  p_name := '&p_name';
25  -- check supplier
26  OPEN cur_supplier;
27  FETCH cur_supplier INTO s_sid, s_snm;
28  IF cur_supplier%NOTFOUND THEN
29      RAISE supplier_not_found;
30  END IF;
31  CLOSE cur_supplier;
32  DBMS_OUTPUT.PUT_LINE('*****');
33  DBMS_OUTPUT.PUT_LINE('PART ID    PART NAME        QUANTITY    UNIT PRICE    TOTAL');
34  DBMS_OUTPUT.PUT_LINE('*****');
35  -- catalog loop
36  OPEN cur_catalog;
37  LOOP
38      FETCH cur_catalog INTO c_pid, c_qty;
39      EXIT WHEN cur_catalog%NOTFOUND;
40      -- parts loop
41      OPEN cur_parts;
42      LOOP
43          FETCH cur_parts INTO p_pid, p_pnm, p_rate;
44          EXIT WHEN cur_parts%NOTFOUND;
45          IF c_pid = p_pid THEN
46              total := c_qty * p_rate;
47              grand_total := grand_total + total;
48              DBMS_OUTPUT.PUT_LINE(RPAD(p_pid,10) || RPAD(p_pnm,14) || RPAD(c_qty,11) || RPAD(p_rate,13) ||
total);
49          END IF;
50      END LOOP;
51      CLOSE cur_parts;
52  END LOOP;
53  CLOSE cur_catalog;

```

```

54      DBMS_OUTPUT.PUT_LINE('*****');
55      DBMS_OUTPUT.PUT_LINE('Grand Total: ' || grand_total);
56  EXCEPTION
57      WHEN supplier_not_found THEN
58          DBMS_OUTPUT.PUT_LINE('Exception: Supplier name not found. ');
59* END;
60 /

```

SQL> /

Enter value for p_name: vishal

old 24: p_name := '&p_name';

new 24: p_name := 'vishal';

Exception: Supplier name not found.

PL/SQL procedure successfully completed.

Enter value for p_name: supplier D

old 24: p_name := '&p_name';

new 24: p_name := 'supplier D';

PART ID	PART NAME	QUANTITY	UNIT PRICE	TOTAL
---------	-----------	----------	------------	-------

P3	MONITOR	30	2500	75000
----	---------	----	------	-------

P1	RAM	50	1000	50000
----	-----	----	------	-------

P2	MOUSE	200	500	100000
----	-------	-----	-----	--------

Grand Total: 225000

PL/SQL procedure successfully completed.

SQL> -- Extra OR my self do that : find total fees [Tables: student, enroll, course]

SQL> ed

Wrote file afiedt.buf

```
1  declare
2      rn_s student.ROLLNO%type;
3      fn student.FNAME%type;  cursor cur_s IS select rollno, fname from student;
4      rn_e enroll.rollno%type;
5      cn_e enroll.courseno%type;
6      cursor cur_e IS select rollno, courseno from enroll where rollno = rn_s;
7      cn_c course.c_no%type;
8      cnm course.coursename%type;
9      fee course.fees%type;
10     cursor cur_c IS select c_no, coursename, fees from course;
11     total_fee number:=0;
12 begin
13     open cur_s;
14     loop
15         total_fee:= 0;
16         fetch cur_s INTO rn_s, fn;
17         exit when cur_s%NOTFOUND;
18         dbms_output.put('RollNumber:' || rn_s || ', First Name: ' || fn || ', total fees: ');
19         open cur_e;
20         loop
21             fetch cur_e INTO rn_e, cn_e;
22             exit when cur_e%NOTFOUND;
23             open cur_c;
24             loop
25                 fetch cur_c INTO cn_c, cnm, fee;
26                 exit when cur_c%NOTFOUND;
27                 if cn_c = cn_e then
28                     total_fee := total_fee + fee;
29                 end if;
30             end loop;
31             close cur_c;
32         end loop;
33     close cur_e;
34     dbms_output.put_line(total_fee);
```

```

35         end loop;
36         close cur_s;
37* end;
SQL> /
RollNumber:11, First Name: Kapil, total fees: 22300
RollNumber:12, First Name: Anjali, total fees: 22300
RollNumber:13, First Name: Ravi, total fees: 5500
RollNumber:14, First Name: Pooja, total fees: 0

```

PL/SQL procedure successfully completed.

```
SQL> -- day-14.docx file exercise
```

```
SQL> -- Topics: Procedure: IN, OUT and INOUT parameters Function, Difference between procedure and function
```

```
SQL> -- 1. Write a procedure which will take Faculty ID as an input and will display all the information of that faculty
```

```
SQL> ed
```

Wrote file afiedt.buf

```

1  create or replace procedure fac_data(n IN number)
2  IS
3      id faculty_master.faculty_id%type;
4      nm faculty_master.faculty_name%type;
5      dj faculty_master.doj%type;
6  BEGIN
7      select faculty_id, faculty_name, doj into id, nm, dj from faculty_master WHERE faculty_id = n;
8      dbms_output.put_line(RPAD(id,5) || RPAD(nm,15) || dj);

```

```
9* end;  
SQL> /
```

Procedure created.

```
SQL> execute fac_data(2);  
2      Prof. Johnson  01-SEP-12
```

PL/SQL procedure successfully completed.

SQL> -- 2. Write a stored procedure that uses an INOUT parameter and an IN parameter. The user will supply 'M' or 'F' through IN parameter (emp_gender) to count a number of male or female from Employee table. The INOUT parameter (mfgender) will return the result to a user.

```
SQL> ed
```

Wrote file afiedt.buf

```
1  create or replace procedure count_gender(emp_gender in char, mfgender in out number)  
2  IS  
3  BEGIN  
4      select count(*) into mfgender from employee where gender = emp_gender;  
5* end;  
SQL> /
```

Procedure created.

```
SQL> ed
```

Wrote file afiedt.buf

```
1  declare  
2      gender_count_result number := 0;  
3  BEGIN  
4      count_gender('F', gender_count_result);  
5      dbms_output.put_line('Female Employees: ' || gender_count_result);  
6* end;  
SQL> /
```

Female Employees: 2

PL/SQL procedure successfully completed.

SQL> ed

Wrote file afiedt.buf

```
1  declare
2      gender_count_result number := 0;
3  BEGIN
4      count_gender('M', gender_count_result);
5      dbms_output.put_line('Male Employees: ' || gender_count_result);
6* end;
```

SQL> /

Male Employees: 4

PL/SQL procedure successfully completed.

SQL> -- 3. Write a procedure which will take minimum limit and maximum limit of salary and the execution of the procedure will display name of the employees having salary between the range.

SQL> ed

Wrote file afiedt.buf

```
1  create or replace procedure emp_salary_range(min in number, max in number)
2  IS
3  BEGIN
4      for i in (select ename, sal from emp where sal between min AND max)
5      loop
6          dbms_output.put_line(RPAD(i.ename, 10) || RPAD(i.sal, 10));
7      end loop;
8* end;
```

SQL> /

Procedure created.

SQL> execute emp_salary_range(1500, 2000);

ALLEN 1600

TURNER 1500

PL/SQL procedure successfully completed.

```
SQL> execute emp_salary_range(1000, 3000);
```

ALLEN	1600
WARD	1250
JONES	2975
MARTIN	1250
BLAKE	2850
CLARK	2450
SCOTT	3000
TURNER	1500
ADAMS	1100
FORD	3000
MILLER	1300

PL/SQL procedure successfully completed.

```
SQL> -- 4. Use following tables and do as directed: Movie (movie_id, movie_name, date_of_release) Screen (screen_id, location, max_capacity), Current (movie_id,screen_id, date_of_arrival, date_of_closure)
```

```
SQL> select * from Movie;
```

MOVIE_ID	MOVIE_NAME	GENRE	RELEASE_D
1	Inception	Sci-Fi	16-JUL-10
2	Avatar	Fantasy	18-DEC-09
3	Interstellar	Sci-Fi	07-NOV-14

```
SQL> select * from Screen;
```

SCREEN_ID	SCREEN_NAME	MAX_CAPACITY
101	Screen 1	80
102	Screen 2	120
103	Screen 3	60

```
SQL> select * from Current_Movie;
```

MOVIE_ID	SCREEN_ID	DATE_OF_A	DATE_OF_C
----------	-----------	-----------	-----------

```
-----
1      101 14-DEC-25
2      102 01-JAN-24
```

SQL> -- 5. Consider the above table and write a function to return the movie name which arrived today.

SQL> ed

Wrote file afiedt.buf

```
1  create or replace function movie_arrived_today
2  Return varchar2
3  IS
4      m_name Movie.movie_name%type;
5  BEGIN
6      select m.movie_name into m_name from movie m join current_movie c on m.movie_id=c.movie_id where
trunc(c.date_of_arrival)=trunc(SYSDATE);
7      return m_name;
8* end;
SQL> /
```

Function created.

SQL> select movie_arrived_today from dual;

MOVIE_ARRIVED_TODAY

Inception

SQL> -- 6. Write a function that will return max_capacity of a screen by providing Screen_Id as a parameter.

SQL> ed

Wrote file afiedt.buf

```
1  create or replace function screen_max_capacity(id in number)
2  return number
3  IS
4      capacity number;
5  BEGIN
6      select max_capacity into capacity from screen where screen_id = id;
```

```
7      return capacity;
8* end;
SQL> /
```

Function created.

```
SQL> select screen_max_capacity(102) from dual;
```

```
SCREEN_MAX_CAPACITY(102)
-----
                        120
```

◆ What is a Procedure?

A **procedure** is a **stored PL/SQL subprogram** used to perform a specific task. It **may or may not return a value** and is generally used to **perform actions**.

◆ Procedure Parameters

1 IN Parameter

- Used to **pass input** to a procedure
- **Default mode**
- Value **cannot be changed** inside procedure

```
CREATE PROCEDURE demo_in (p_id IN NUMBER)
```

2 OUT Parameter

- Used to **return a value**
- Value must be **assigned inside** procedure
- Initial value from caller is **ignored**

```
CREATE PROCEDURE demo_out (p_total OUT NUMBER)
```

3 IN OUT Parameter

- Used to **pass input and return output**
- Value can be **read and modified**

```
CREATE PROCEDURE demo_inout (p_count IN OUT NUMBER)
```

◆ Example Procedure with IN & OUT

```
CREATE OR REPLACE PROCEDURE get_salary ( p_empno IN NUMBER, p_sal OUT NUMBER )
IS
BEGIN
    SELECT sal INTO p_sal FROM emp WHERE empno = p_empno;
END;
/
```

◆ How to Call a Procedure (IN / OUT / INOUT)

```
DECLARE
    v_sal NUMBER;
BEGIN
    get_salary(101, v_sal);
    DBMS_OUTPUT.PUT_LINE(v_sal);
END;
/
```

◆ What is a Function?

A **function** is a **stored PL/SQL subprogram** that **must return a value** using the RETURN statement.

👉 Functions are mainly used in **expressions and SQL queries**.

◆ Example Function

```
CREATE OR REPLACE FUNCTION get_bonus ( p_sal NUMBER )
RETURN NUMBER
IS
BEGIN
    RETURN p_sal * 0.10;
END;
/
```

◆ How to Call a Function

```
SELECT get_bonus(30000) FROM dual;
```

◆ Difference between Procedure and Function

Feature	Procedure	Function
Return value	Optional	Mandatory

Feature	Procedure	Function
Called using	EXEC / PL/SQL block	SELECT ... FROM dual
Used in SQL	✗ No	✓ Yes
Purpose	Perform action	Compute and return value
Parameters	IN, OUT, INOUT	Mostly IN
RETURN keyword	Optional	Required

◆ Key Viva Points (Very Important)

- Procedure → **action-oriented**
- Function → **value-oriented**
- Function **must return exactly one value**
- OUT / INOUT parameters **cannot be literals**
- Functions **cannot perform DML** when called from SQL

★ Viva-Ready Short Answer (MEMORIZE)

A procedure is a stored PL/SQL program used to perform a task and may return values using OUT or INOUT parameters. A function is a stored PL/SQL program that must return a single value and can be used in SQL expressions. IN parameters are used for input, OUT parameters for output, and INOUT parameters for both input and output.

```
SQL> -- day-15.docx file exercise
```

```
SQL> -- Topics: Explain Triggers. Use of Trigger Types of Triggers
```

```
SQL> -- 1. Create a trigger that when the new record is inserted in the Movie table
```

```
SQL> ed
```

```
Wrote file afiedt.buf
```

```
1 create or replace trigger movie_insert_trigger
```

```
2 AFTER insert ON movie
3 FOR EACH ROW
4 BEGIN
5     dbms_output.put_line('New Movie Inserted: ' || :NEW.movie_name);
6* end;
SQL> /
```

Trigger created.

```
SQL> INSERT INTO Movie VALUES (4, 'Dune', 'Sci-Fi', SYSDATE-1);
New Movie Inserted: Dune
```

1 row created.

```
SQL> -- 2. Create a trigger that will be fired when a Movie is deleted
```

```
SQL> ed
```

Wrote file afiedt.buf

```
1 create or replace trigger movie_delete_trigger
2 AFTER delete ON movie
3 FOR EACH ROW
4 BEGIN
5     dbms_output.put_line('Movie Deleted: ' || :OLD.movie_name);
6* end;
SQL> /
```

Trigger created.

```
SQL> delete from movie where movie_id=4;
Movie Deleted: Dune
```

1 row deleted.

```
SQL> -- 3. Create a trigger when the max_capacity of the screen is more than 100.
```

```
SQL> ed
```

Wrote file afiedt.buf

```
1  create or replace trigger max_capacity_trigger
2  AFTER insert OR update ON screen
3  FOR EACH ROW
4  BEGIN
5      if :NEW.max_capacity > 100 then
6          dbms_output.put_line('Warning: Screen capacity greater than 100');
7      end if;
8* end;
SQL> /
```

Trigger created.

```
SQL> insert into screen values (104, 'Screen 4', 150);
Warning: Screen capacity greater than 100
```

1 row created.

◆ What is a Trigger?

A **trigger** is a **database object** that is **automatically executed** when a specific event occurs on a table or view.

👉 Triggers **fire automatically** – they are **not called explicitly**.

◆ Why Triggers are used? (Use of Triggers)

Triggers are used to:

1. **Enforce business rules**
2. **Maintain data integrity**
3. **Audit or log changes**
4. **Validate data before insert/update**
5. **Perform automatic actions**

📌 Example:

- Prevent salary from being negative
 - Log deleted records
 - Validate capacity before insert
-

◆ Syntax of Trigger (Basic)

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE | AFTER
INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
    -- trigger body
END;
/
```

◆ Trigger Timing

1 BEFORE Trigger

- Executes **before** DML operation
- Used for **validation**

📌 Example: BEFORE INSERT ON table

2 AFTER Trigger

- Executes **after** DML operation
- Used for **logging / auditing**

📌 Example: AFTER DELETE ON table

◆ Trigger Level

1 Row-Level Trigger

- Executes **once per row**
- Uses :NEW and :OLD

📌 Example: FOR EACH ROW

2 Statement-Level Trigger

- Executes **once per SQL statement**

- Does NOT use :NEW or :OLD

◆ Trigger Events (Types of Triggers)

◆ DML Triggers

Fired on:

- INSERT
- UPDATE
- DELETE

✦ Example: AFTER INSERT ON employee

◆ DDL Triggers

Fired on:

- CREATE
- ALTER
- DROP

✦ Example: AFTER CREATE ON SCHEMA

◆ Database Triggers

Fired on database events:

- LOGON
- LOGOFF
- STARTUP
- SHUTDOWN

✦ Example: AFTER LOGON ON DATABASE

◆ :NEW and :OLD Keywords

Operation	:OLD	:NEW
INSERT	✗	✓
UPDATE	✓	✓
DELETE	✓	✗

◆ Example Trigger (Simple)

CREATE OR REPLACE TRIGGER trg_salary_check

```
BEFORE INSERT OR UPDATE ON employee
FOR EACH ROW
BEGIN
    IF :NEW.salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be negative');
    END IF;
END;
/
```

◆ **Advantages of Triggers**

- ✓ Automatic execution
- ✓ Centralized business logic
- ✓ Improved data integrity

◆ **Disadvantages of Triggers**

- ✗ Hard to debug
- ✗ Hidden execution
- ✗ Can reduce performance

★ **Viva-Ready Short Answer (MEMORIZE THIS)**

A trigger is a database object that automatically executes when a specific event occurs on a table or view. Triggers are used for validation, auditing, and enforcing business rules. Types of triggers include DML, DDL, and database triggers, and they can be BEFORE or AFTER, row-level or statement-level.